PrepNPlaced
prepnplaced.com

DATA ANALYTICS + DATA ENGINEERING

SQL END TO END NOTES

BASIC TO ADVANCED, IN ONE FLOW

From your first SELECT to the patterns interviewers keep reusing. Every topic comes with the approach to solve it, a query that runs, and the real output.

40	5	18	38
PAGES	PARTS	INTERVIEW PATTERNS	SOLVED SQL QUESTIONS

`SELECT` , `JOIN` , `GROUP BY` , `WINDOW` , `RECURSIVE` , `PATTERNS` , `MERGE` , `SCD 2`

Written and tested on MySQL 8, with dialect notes for

MySQL · PostgreSQL · Snowflake · Databricks · BigQuery · Spark SQL

top_2_per_dept.sql

```sql
WITH ranked AS (
  SELECT dept, name, salary,
    DENSE_RANK() OVER (
      PARTITION BY dept
      ORDER BY salary DESC) AS rnk
  FROM employees
)
SELECT dept, name, salary
FROM ranked
WHERE rnk <= 2;
```

REAL OUTPUT, MYSQL 8.0

dept	name	salary
Analytics	Ishita	160000
Analytics	Neha	150000
Engineering	Anita	250000
Engineering	Rahul	180000
Sales	Dev	90000
Sales	Tara	90000

Ties stay tied. Dev and Tara both earn 90,000, so DENSE_RANK keeps both and Sales returns two people. Page 21 explains when you want that.

THE ROUTE

01 **Foundations** — SELECT to subqueries
02 **Advanced SQL** — windows, recursion, pivots
03 **Interview patterns** — 18 patterns, solved
04 **SQL for data engineering** — loads, MERGE, SCD 2, DQ
05 **Practice and revision** — DA + DE sets, cheat sheet

START HERE

WHAT IS INSIDE

One continuous path. Each page builds on the one before it, and every query runs on the same small store dataset.

START HERE

Start here: the running dataset 03
The 6-step approach to any SQL question 04

PART 1 · FOUNDATIONS

How a query actually runs 05
Filtering rows and the NULL trap 06
CASE, string and date functions 07
Aggregation: GROUP BY and HAVING 08
Joins: the six you need 09
Join traps interviewers set 10
Subqueries, EXISTS and the NOT IN trap 11
CTEs and set operations 12
Practice set 1: foundations 13

PART 2 · ADVANCED SQL

Window functions 1: ranking 14
Window functions 2: frames, running totals 15
LAG, LEAD and value functions 16
Recursive CTEs: hierarchies and series 17
Pivot, unpivot and subtotals 18
Practice set 2: advanced 19

PART 3 · INTERVIEW PATTERNS

The pattern map: 18 patterns at a glance 20
P1 Top-N per group · P2 Nth highest 21
P3 Deduplicate · P4 Running total, % of total 22
P5 Rolling window · P6 Period over period 23
P7 Gaps and islands: streaks 24
P8 Sessionization · P9 First and Nth event 25
P10 Cohort retention 26
P11 Funnel conversion · P12 Rates and ratios 27
P13 Never did X, did all of X · P14 Row vs related row 28
P15 Median and percentiles · P16 Overlapping intervals 29
P17 Pairs bought together · P18 Fill missing dates 30

PART 4 · SQL FOR DATA ENGINEERING

DDL, keys and data modelling 31
Indexes, EXPLAIN and query performance 32
Transactions, ACID and isolation levels 33
Incremental loads, upserts and MERGE 34
Slowly changing dimensions: SCD 1 and 2 35
Data quality checks and reconciliation 36

PART 5 · PRACTICE AND REVISION

Interview set: data analytics 37
Interview set: data engineering 38
Concept questions with short answers 39
Revision sheet and 4-week practice plan 40

HOW TO USE THIS BOOK

If you are preparing for a **Data Analyst** role, read Parts 1 to 3 in order, then Part 5. If you are preparing for **Data Engineering**, do the same and add Part 4. Interviewers for DE roles still open with the analyst-style questions, then move to loads, upserts, SCDs and performance.

How every page is built

APPROACH
How to think before you type. The steps, in order.

TRAP
A query that runs and returns wrong rows. Learn these.

INTERVIEW TIP
What to say out loud. The line interviewers listen for.

DATA ENGINEERING NOTE
How the idea changes at warehouse scale.

REMEMBER
A rule worth memorising, or a dialect difference.

Code blocks marked **MySQL 8** run exactly as printed against the store dataset, and their result tables come straight from MySQL. Blocks marked with another engine (Snowflake, Databricks, PostgreSQL) show syntax MySQL does not have.

START HERE

ONE DATASET, START TO FINISH

Every query in this book runs against the same small online-store database. Small on purpose: you can check each output by eye.

These notes go from your first SELECT to the questions that decide DA and DE interviews. The pattern section comes from the questions I keep seeing across the 20+ analytics and data engineering interviews I have sat, and the ones my mentees report back from their own loops. Each result table you see was produced by running the query on **MySQL 8.0** against the data below, so the numbers are real, not typed in. The full setup script ships with this PDF as `store_dataset.sql`: run it once and every query in the book works on your machine.

The tables

customers 8 rows

customer_id PK	INT
name	VARCHAR
city	VARCHAR NULL
signup_date	DATE
referred_by	INT NULL

orders 15 rows

order_id PK	INT
customer_id	INT
order_date	DATE
status	VARCHAR
amount	DECIMAL

order_items 24 rows

order_id PK	INT
product_id PK	INT
qty	INT
unit_price	DECIMAL

products 8 rows

product_id PK	INT
product_name	VARCHAR
category	VARCHAR
price	DECIMAL

employees 9 rows

emp_id PK	INT
name	VARCHAR
dept	VARCHAR
manager_id	INT NULL
salary	INT
hire_date	DATE

logins 12 rows

user_id PK	INT
login_date PK	DATE

events 12 rows

user_id	INT
event_time	DATETIME
event_type	VARCHAR

orders_raw 6 rows

order_id	INT
status	VARCHAR
amount	DECIMAL
ingested_at	DATETIME

Plus `bookings` (room stays, used for overlapping intervals on page 29). Keys that join: `orders.customer_id` to `customers`, `order_items.order_id` to `orders`, `order_items.product_id` to `products`, `employees.manager_id` to `employees.emp_id` and `customers.referred_by` to `customers.customer_id`.

A first look at the data

ORDERS (ALL 15 ROWS)

order_id	cust	order_date	status	amount
101	1	2025-01-10	delivered	1198
102	2	2025-01-15	delivered	796
103	1	2025-02-02	delivered	1499
104	3	2025-02-10	cancelled	1999
105	2	2025-02-18	delivered	1348
106	4	2025-03-01	delivered	1048
107	1	2025-03-05	returned	798
108	5	2025-03-12	delivered	3498
109	3	2025-03-20	delivered	747
110	6	2025-03-22	delivered	1998
111	2	2025-04-02	delivered	1999
112	7	2025-04-05	delivered	846
113	4	2025-04-15	cancelled	1499
114	1	2025-04-20	delivered	2397
115	5	2025-04-28	delivered	1598

CUSTOMERS (ALL 8 ROWS)

id	name	city	signup_date	ref
1	Aarav	Bengaluru	2025-01-05	NULL
2	Diya	Mumbai	2025-01-12	1
3	Kabir	Delhi	2025-02-03	NULL
4	Meera	Bengaluru	2025-02-20	2
5	Rohan	Pune	2025-03-08	NULL
6	Sana	NULL	2025-03-15	NULL
7	Vikram	Delhi	2025-04-01	3
8	Zoya	Hyderabad	2025-04-10	NULL

PLANTED TRAPS

Sana has no city and Zoya has never ordered. Both are planted on purpose. Many of the traps in this book trace back to those two rows.

DA asked in analyst rounds DE asked in engineering rounds EASY MEDIUM HARD difficulty

START HERE

THE 6-STEP APPROACH

Use this on every question in this book. Interviewers grade the way you get there almost as much as the query.

01 Fix the grain
Say out loud what one output row is. "One row per city" or "one row per customer per month".

02 Map the tables
Which tables, which keys, and is each join 1:1 or 1:many? A 1:many join multiplies rows.

03 Sketch output
Write two or three rows of the result you expect, with column names, before any SQL.

04 Name the pattern
Top-N, running total, islands, anti-join... the pattern map on page 20 lists 18 of them.

05 Build in layers
One CTE per idea. Run each layer, check the row count, then add the next one.

06 Break it
NULLs, ties, duplicates, empty groups, date edges. Then mention indexes or partitions.

Watch it work on one question

Q1. For each city, find the customer who spent the most on delivered orders. Show ties. DA MEDIUM

Step	What you say in the interview
01 Grain	One row per city per top customer. Ties mean a city can return two rows.
02 Tables	`customers` 1 : many `orders` on customer_id. Filter `status = 'delivered'` before summing.
03 Output	`city | name | total_spent`, for example `Bengaluru | Aarav | 5094`.
04 Pattern	Aggregate first, then top-N per group (page 21) with RANK so ties survive.
05 Layers	CTE `spend`: total per customer. CTE `ranked`: RANK per city. Final: keep rank 1.
06 Break it	Sana has a NULL city: PARTITION BY puts NULLs in their own group, so she shows up as city NULL. Say whether you keep or drop her.

SOLUTION MYSQL 8

```
WITH spend AS (                -- layer 1: grain = customer
  SELECT c.customer_id, c.name, c.city,
         SUM(o.amount) AS total_spent
  FROM customers c
  JOIN orders o ON o.customer_id = c.customer_id
  WHERE o.status = 'delivered'
  GROUP BY c.customer_id, c.name, c.city
),
ranked AS (                    -- layer 2: rank inside city
  SELECT *, RANK() OVER (PARTITION BY city
                         ORDER BY total_spent DESC) AS rnk
  FROM spend
)
SELECT city, name, total_spent  -- layer 3: keep the top
FROM ranked
WHERE rnk = 1
ORDER BY total_spent DESC;
```

RESULT

city	name	total_spent
Pune	Rohan	5096
Bengaluru	Aarav	5094
Mumbai	Diya	4143
NULL	Sana	1998
Delhi	Vikram	846

INTERVIEW TIP
Run layer 1 alone first and say the row count out loud: "7 customers have delivered orders." It shows the interviewer you test as you build, and it catches fan-out early.

APPROACH
When you get stuck: drop back to step 3. Most wrong answers come from a fuzzy idea of the output, not from syntax. If the expected output is clear, the pattern usually names itself.

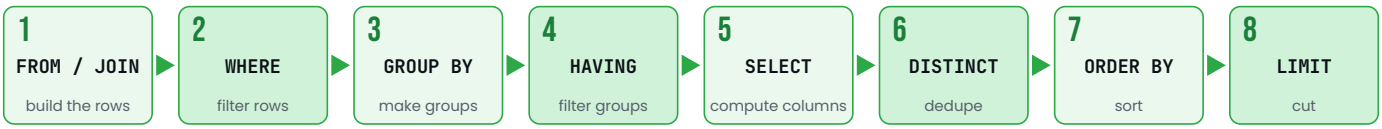
PART 1 · FOUNDATIONS · CHAPTER 1

HOW A QUERY ACTUALLY RUNS

01

You write SELECT first, but the database reads it almost last. Most beginner errors make sense once you know the order.

Written order vs logical order



What the order explains

- `WHERE` cannot use `SUM()` or `COUNT()`: groups do not exist yet at step 2. Use `HAVING`.
- `WHERE` cannot use a SELECT alias: the alias is born at step 5. MySQL lets you use it in GROUP BY, HAVING and ORDER BY; PostgreSQL only in ORDER BY (and GROUP BY).
- Window functions run at step 5, so you cannot filter them in WHERE. Wrap them in a CTE, or use `QUALIFY` in Snowflake, BigQuery and Databricks.
- `ORDER BY` can use an alias because it runs after SELECT.
- `LIMIT` without `ORDER BY` returns any N rows. Not "the first N".

EVERY CLAUSE, ONE QUERY — MYSQL 8

```sql
SELECT status,
       COUNT(*)    AS orders,
       SUM(amount) AS revenue
FROM orders
WHERE order_date >= '2025-02-01'   -- step 2
GROUP BY status                    -- step 3
HAVING COUNT(*) > 1                -- step 4
ORDER BY revenue DESC;             -- step 7
```

RESULT

status	orders	revenue
delivered	10	16978
cancelled	2	3498

The five sublanguages

Group	Commands	What it does	Asked as
DQL	`SELECT`	Reads data	Every query question
DML	`INSERT UPDATE DELETE MERGE`	Changes rows	"Write an upsert", "delete duplicates"
DDL	`CREATE ALTER DROP TRUNCATE`	Changes structure	DELETE vs TRUNCATE vs DROP
DCL	`GRANT REVOKE`	Permissions	Rare in DA, sometimes in DE
TCL	`BEGIN COMMIT ROLLBACK SAVEPOINT`	Transactions	ACID, page 33

Data types that come up in interviews

Type	Use it for	Watch out for
`INT / BIGINT`	IDs, counts	INT tops out near 2.1 billion. Event tables blow past that; use BIGINT for surrogate keys.
`DECIMAL(10,2)`	Money	FLOAT/DOUBLE store approximations. 0.1 + 0.2 is not exactly 0.3. Never store rupees in FLOAT.
`VARCHAR(n) / CHAR(n)`	Text	CHAR pads to n. VARCHAR stores what you give it. Use CHAR only for fixed codes like country ISO.
`DATE / DATETIME / TIMESTAMP`	Time	MySQL TIMESTAMP is stored in UTC and converted to the session time zone; DATETIME is stored as written.
`BOOLEAN`	Flags	MySQL BOOLEAN is TINYINT(1) under the hood, so TRUE = 1.

Q2. Why does the first query fail, and how do you fix it? DA DE EASY

FAILS — MYSQL 8

```sql
SELECT order_id, amount * 0.18 AS gst
FROM orders
WHERE gst > 100;
-- ERROR 1054: Unknown column 'gst'
--               in 'where clause'
```

WORKS — MYSQL 8

```sql
SELECT order_id, gst
FROM (SELECT order_id,
             amount * 0.18 AS gst
      FROM orders) t
WHERE gst > 100;   -- or repeat the expression
```

WHERE runs at step 2 and the alias is created at step 5, so WHERE has never heard of gst. Wrap the SELECT in a subquery or CTE so the alias exists before the filter.

PART 1 · FOUNDATIONS · CHAPTER 2

FILTERING AND THE NULL TRAP

02

WHERE keeps a row only when the condition is TRUE. Not FALSE, and not UNKNOWN. That one rule causes most NULL bugs.

Operator	Example	Note
`= <> < > <= >=`	`amount >= 1000`	`<>` and `!=` mean the same thing
`AND OR NOT`	`status = 'delivered' AND amount > 1000`	AND binds tighter than OR. Use brackets.
`IN (...)`	`city IN ('Delhi','Pune')`	Short for several ORs
`BETWEEN a AND b`	`order_date BETWEEN '2025-03-01' AND '2025-03-31'`	Inclusive on both ends
`LIKE`	`name LIKE 'A%' name LIKE '_a%'`	`%` = any run of characters, `_` = exactly one
`IS NULL / IS NOT NULL`	`city IS NULL`	`= NULL` never matches anything

Three-valued logic: TRUE, FALSE and UNKNOWN

Any comparison with NULL gives UNKNOWN, not FALSE. `NULL = NULL` is UNKNOWN. `NULL <> 'Mumbai'` is UNKNOWN. WHERE drops UNKNOWN rows silently.

A	B	A AND B	A OR B
TRUE	UNKNOWN	**UNKNOWN**	**TRUE**
FALSE	UNKNOWN	**FALSE**	**UNKNOWN**
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN
NOT UNKNOWN		**UNKNOWN**	

REMEMBER

FALSE AND anything is FALSE. TRUE OR anything is TRUE. Everything else that touches NULL stays UNKNOWN.

CUSTOMERS OUTSIDE MUMBAI? — MYSQL 8

```
SELECT name, city
FROM customers
WHERE city <> 'Mumbai';
```

RESULT

name	city
Aarav	Bengaluru
Kabir	Delhi
Meera	Bengaluru
Rohan	Pune
Vikram	Delhi
Zoya	Hyderabad

6 rows. Sana (city NULL) is gone, though she is clearly not in Mumbai.

THREE WAYS TO KEEP THE NULL ROW — MIXED

```
-- portable fix
WHERE city <> 'Mumbai' OR city IS NULL

-- MySQL null-safe equality operator
WHERE NOT (city <=> 'Mumbai')

-- PostgreSQL / Snowflake / Databricks
WHERE city IS DISTINCT FROM 'Mumbai'
```

Functions that tame NULL

- `COALESCE(city, 'Unknown')` returns the first non-NULL argument. ANSI, works everywhere.
- `IFNULL(a, b)` is MySQL's two-argument version. `NVL` in Oracle and Snowflake.
- `NULLIF(a, b)` returns NULL when a = b. The classic use: `x / NULLIF(y, 0)` so a zero divisor gives NULL instead of an error.
- `NULL + 5` is NULL. `CONCAT('a', NULL)` is NULL in MySQL. Wrap with COALESCE first.

PRECEDENCE TRAP: WHAT DOES THIS RETURN? — MYSQL 8

```
SELECT order_id, status, amount
FROM orders
WHERE status = 'cancelled' OR status = 'returned'
  AND amount > 1000;
```

RESULT

order_id	status	amount
104	cancelled	1999
113	cancelled	1499

TRAP

AND runs before OR, so this reads as `cancelled OR (returned AND amount > 1000)`. Order 113 is cancelled at 1,499 and order 104 at 1,999, both kept. The returned order (798) fails the amount test. If you meant "cancelled or returned, and over 1,000", write `WHERE status IN ('cancelled','returned') AND amount > 1000`.

INTERVIEW TIP

BETWEEN on a DATETIME column with a date literal stops at midnight: `'2025-03-31'` means 2025-03-31 00:00:00. For timestamps use a half-open range: `ts >= '2025-03-01' AND ts < '2025-04-01'`.

PART 1 · FOUNDATIONS · CHAPTER 3

CASE, STRINGS AND DATES

03

CASE is SQL's if-else. Strings and dates are where dialects differ the most, so learn one well and know the others exist.

SEARCHED CASE: FIRST TRUE WHEN WINS — MYSQL 8

```sql
SELECT order_id, amount,
  CASE
    WHEN amount >= 2000 THEN 'large'
    WHEN amount >= 1000 THEN 'medium'
    ELSE 'small'
  END AS bucket,
  DATE_FORMAT(order_date, '%Y-%m') AS month
FROM orders
WHERE status = 'delivered'
ORDER BY amount DESC
LIMIT 5;
```

RESULT

order_id	amount	bucket	month
108	3498	large	2025-03
114	2397	large	2025-04
111	1999	medium	2025-04
110	1998	medium	2025-03
115	1598	medium	2025-04

REMEMBER

The first TRUE branch wins, so order matters. With no ELSE, unmatched rows get NULL.

String functions

MySQL	Does	PostgreSQL / Snowflake
`CONCAT(a, ' ', b)`	Join strings (NULL if any part is NULL)	`a || ' ' || b`
`CONCAT_WS('-', a, b)`	Join with a separator, skips NULLs	`CONCAT_WS('-', a, b)`
`UPPER(s) LOWER(s) TRIM(s)`	Case and whitespace	same
`SUBSTRING(s, 2, 3)`	3 characters starting at position 2 (1-based)	`SUBSTRING(s, 2, 3)` / `SUBSTR`
`CHAR_LENGTH(s)`	Characters. LENGTH() counts bytes in MySQL	`LENGTH(s)`
`LOCATE('@', email)`	Position of a substring, 0 if absent	`POSITION('@' IN email)`
`SUBSTRING_INDEX(email, '@', -1)`	Text after the last @ (email domain)	`SPLIT_PART(email, '@', 2)`

Date functions

MySQL	Does	PostgreSQL	Snowflake / Databricks
`CURDATE() NOW()`	Today, now	`CURRENT_DATE NOW()`	`CURRENT_DATE()`
`DATE_FORMAT(d, '%Y-%m')`	Month label	`TO_CHAR(d, 'YYYY-MM')`	`TO_CHAR / DATE_FORMAT`
`DATE_FORMAT(d, '%Y-%m-01')`	First of month	`DATE_TRUNC('month', d)`	`DATE_TRUNC('month', d)`
`YEAR(d) MONTH(d) DAYOFWEEK(d)`	Parts	`EXTRACT(YEAR FROM d)`	`YEAR(d) EXTRACT`
`DATEDIFF(d2, d1)`	Days between	`d2 - d1`	`DATEDIFF('day', d1, d2)`
`DATE_ADD(d, INTERVAL 7 DAY)`	Shift a date	`d + INTERVAL '7 days'`	`DATEADD('day', 7, d)`
`TIMESTAMPDIFF(MINUTE, t1, t2)`	Gap in any unit	`EXTRACT(EPOCH FROM t2-t1)/60`	`DATEDIFF('minute', t1, t2)`

DATE MATH — MYSQL 8

```sql
SELECT name, signup_date,
  DATEDIFF('2025-04-30', signup_date) AS days_since,
  DAYNAME(signup_date) AS weekday
FROM customers
WHERE signup_date >= '2025-03-01';
```

RESULT

name	signup_date	days_since	weekday
Rohan	2025-03-08	53	Saturday
Sana	2025-03-15	46	Saturday
Vikram	2025-04-01	29	Tuesday
Zoya	2025-04-10	20	Thursday

TRAP

Argument order flips between engines. MySQL `DATEDIFF(end, start)`; SQL Server and Snowflake `DATEDIFF(unit, start, end)`. In an interview, say which engine you are writing for before you write the first date function.

PART 1 · FOUNDATIONS · CHAPTER 4

AGGREGATION: GROUP BY AND HAVING

04

GROUP BY folds many rows into one row per group. Every column you SELECT is either in the GROUP BY or inside an aggregate.

THREE DIFFERENT COUNTS — MYSQL 8

```
SELECT COUNT(*)              AS all_rows,
       COUNT(city)           AS has_city,
       COUNT(DISTINCT city)  AS cities,
       COUNT(referred_by)    AS referred
FROM customers;
```

RESULT

all_rows	has_city	cities	referred
8	7	5	3

- `COUNT(*)` counts rows, NULLs included.
- `COUNT(col)` skips rows where col is NULL. Sana has no city, so 7.
- `COUNT(DISTINCT col)` counts unique non-NULL values: Bengaluru, Mumbai, Delhi, Pune, Hyderabad.
- `COUNT(1)` is the same as `COUNT(*)`. No speed difference in modern engines.
- `SUM AVG MIN MAX` all ignore NULLs. `AVG(col)` divides by non-NULL rows only, which can quietly inflate an average.

WHERE filters rows, HAVING filters groups

CITIES WITH 2+ DELIVERED ORDERS — MYSQL 8

```
SELECT c.city,
       COUNT(*)      AS delivered_orders,
       SUM(o.amount) AS revenue,
       ROUND(AVG(o.amount), 2) AS avg_order
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'delivered'        -- rows first
GROUP BY c.city
HAVING COUNT(*) >= 2                -- then groups
ORDER BY revenue DESC;
```

RESULT

city	delivered_orders	revenue	avg_order
Bengaluru	4	6142	1535.50
Pune	2	5096	2548
Mumbai	3	4143	1381
Delhi	2	1593	796.50

Sana's single order sits alone in the NULL-city group, so HAVING drops it. Without the WHERE, Delhi would also count Kabir's cancelled order and show 3.

Conditional aggregation: many metrics in one pass

CASE INSIDE SUM — MYSQL 8

```
SELECT customer_id,
  COUNT(*) AS orders,
  SUM(CASE WHEN status = 'delivered'
           THEN amount ELSE 0 END) AS delivered_amt,
  SUM(status = 'cancelled') AS cancelled
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

RESULT

customer_id	orders	delivered_amt	cancelled
1	4	5094	0
2	3	4143	0
3	2	747	1
4	2	1048	1
5	2	5096	0
6	1	1998	0
7	1	846	0

REMEMBER

`SUM(status = 'cancelled')` works in MySQL because a comparison returns 1 or 0. Portable form: `SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END)`. PostgreSQL also has `COUNT(*) FILTER (WHERE ...)`.

TRAP

ONLY_FULL_GROUP_BY. `SELECT city, name, COUNT(*) FROM customers GROUP BY city` fails in MySQL 8 and PostgreSQL: which name should a group of two customers show? Old MySQL silently picked an arbitrary one. Either group by name too, aggregate it (`MAX(name)`, `GROUP_CONCAT(name)`), or use a window function.

The classic: find duplicate rows

GROUP BY EVERY COLUMN, KEEP GROUPS > 1 — MYSQL 8

```
SELECT order_id, status, ingested_at,
       COUNT(*) AS copies
FROM orders_raw
GROUP BY order_id, status, amount, ingested_at
HAVING COUNT(*) > 1;
```

RESULT

order_id	status	ingested_at	copies
103	placed	2025-02-02 08:15	2

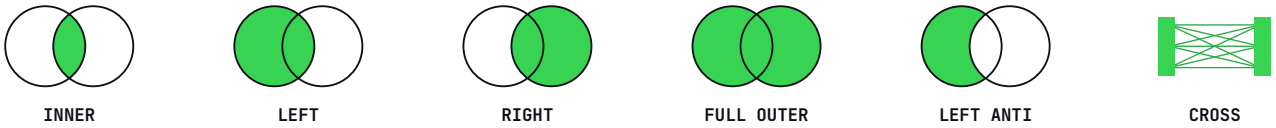
Order 103 was ingested twice with identical values. Group by the columns that define "the same row". To delete the extra copy and keep one, you need ROW_NUMBER, which is pattern P3 on page 22.

PART 1 · FOUNDATIONS · CHAPTER 5

JOINS: THE SIX YOU NEED

05

A join decides which rows survive when two tables meet. Know what each one keeps, and how many rows it should return.



Join	Keeps	customers x orders	Typical use
`INNER JOIN`	Only rows that match on both sides	15 rows	Orders with their customer
`LEFT JOIN`	Every left row; NULLs where the right has no match	16 rows (Zoya added)	Keep all customers, even with no orders
`RIGHT JOIN`	Mirror of LEFT. Rewrite as LEFT by swapping tables	15 rows	Rare; most teams ban it for readability
`FULL OUTER JOIN`	Everything from both sides	not in MySQL	Reconciling two sources (DE)
`LEFT ANTI` (pattern)	Left rows with no match: LEFT JOIN + IS NULL	1 row (Zoya)	"Customers who never ordered"
`CROSS JOIN`	Every pair. Rows = left x right	64 (customers x products)	Calendars, grids, all combinations

LEFT JOIN KEEPS ZOYA WITH NULLS — MYSQL 8

```
SELECT c.name, o.order_id, o.status
FROM customers c
LEFT JOIN orders o
       ON o.customer_id = c.customer_id
WHERE c.customer_id IN (6, 7, 8)
ORDER BY c.customer_id;
```

RESULT

name	order_id	status
Sana	110	delivered
Vikram	112	delivered
Zoya	NULL	NULL

FULL OUTER JOIN, MYSQL STYLE — MYSQL 8

```
-- MySQL has no FULL OUTER JOIN. Emulate it:
SELECT a.id, a.amt AS src_amt, b.amt AS tgt_amt
FROM src a LEFT JOIN tgt b ON b.id = a.id
UNION ALL
SELECT b.id, NULL, b.amt
FROM tgt b LEFT JOIN src a ON a.id = b.id
WHERE a.id IS NULL;   -- only rows missing from src
```

DATA ENGINEERING NOTE

In PostgreSQL, Snowflake, BigQuery and Databricks just write `FULL OUTER JOIN`. Use it to compare yesterday's load with today's, and the source system with the warehouse.

Self join: a table joined to itself

EMPLOYEE AND MANAGER NAMES — MYSQL 8

```
SELECT e.name AS employee,
       m.name AS manager
FROM employees e
LEFT JOIN employees m
       ON m.emp_id = e.manager_id
WHERE e.dept = 'Analytics';
```

RESULT

employee	manager
Neha	Anita
Arjun	Neha
Ishita	Neha

Same table, two aliases. LEFT JOIN so Anita, who has no manager, would still appear if her department were selected.

Q3. Table A has values (1, 1, 2, NULL). Table B has (1, 1, 1, NULL, 3). How many rows does each join return on A.v = B.v?

DA DE MEDIUM

PROVE IT — MYSQL 8

```
WITH a AS (SELECT 1 AS v UNION ALL SELECT 1
           UNION ALL SELECT 2 UNION ALL SELECT NULL),
     b AS (SELECT 1 AS v UNION ALL SELECT 1 UNION ALL SELECT 1
           UNION ALL SELECT NULL UNION ALL SELECT 3)
SELECT (SELECT COUNT(*) FROM a JOIN b ON a.v = b.v)       AS inner_j,
       (SELECT COUNT(*) FROM a LEFT JOIN b ON a.v = b.v)  AS left_j,
       (SELECT COUNT(*) FROM a RIGHT JOIN b ON a.v = b.v) AS right_j,
       (SELECT COUNT(*) FROM a CROSS JOIN b)              AS cross_j;
```

RESULT

inner_j	left_j	right_j	cross_j
6	8	8	20

Inner 6: the two 1s in A each match three 1s in B. 2, 3 and both NULLs match nothing. **Left 8:** 6 + A's unmatched 2 and NULL. **Right 8:** 6 + B's NULL and 3. **Full 10:** left + right - inner. **Cross 20:** 4 x 5.

PART 1 · FOUNDATIONS · CHAPTER 6

JOIN TRAPS INTERVIEWERS SET

06

These four mistakes produce queries that run fine and return wrong numbers. That makes them the favourite follow-up questions.

Trap 1: a WHERE filter turns your LEFT JOIN into an INNER JOIN

WRONG: 7 CUSTOMERS — MYSQL 8

```
SELECT c.name, COUNT(o.order_id) AS delivered
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
GROUP BY c.name;
```

RIGHT: ALL 8 CUSTOMERS — MYSQL 8

```
SELECT c.name, COUNT(o.order_id) AS delivered
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
                  AND o.status = 'delivered'
GROUP BY c.name;
```

For Zoya, the LEFT JOIN produces a row with `o.status = NULL`. WHERE then tests `NULL = 'delivered'`, gets UNKNOWN, and throws her out, so the report silently loses every customer with zero delivered orders. **Rule:** conditions on the right table of a LEFT JOIN go in `ON`. Conditions on the left table go in `WHERE`.

Trap 2: fan-out double counts your revenue

WRONG: RETURNS 34498 — MYSQL 8

```
SELECT SUM(o.amount) AS revenue
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
WHERE o.status = 'delivered';
```

An order with 3 items appears 3 times after the join, so its amount is added 3 times. True delivered revenue is **18972**.

RIGHT: 18972 — MYSQL 8

```
-- aggregate to the order grain first
WITH items AS (
  SELECT order_id, COUNT(*) AS n_lines
  FROM order_items GROUP BY order_id
)
SELECT SUM(o.amount) AS revenue,
       SUM(i.n_lines) AS item_lines
FROM orders o
JOIN items i ON i.order_id = o.order_id
WHERE o.status = 'delivered';
```

APPROACH

Before joining, ask "what is the grain of each table?" orders is one row per order, order_items is one row per order per product. Joining a 1-row grain to a many-row grain repeats the 1-row side. Aggregate the many side down first, then join.

Trap 3 and 4: semi joins, anti joins and NULL keys

SEMI AND ANTI JOIN — MYSQL 8

```
-- semi join: customers with at least one order
SELECT c.name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.customer_id = c.customer_id);

-- anti join: customers with no orders
SELECT c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

- A plain JOIN to orders would return Aarav 4 times. `EXISTS` returns each customer once, no DISTINCT needed.
- Anti join: test a column that is **never NULL** in the right table, like its primary key. Testing `o.status IS NULL` breaks if status can be NULL.
- `NULL = NULL` is not true, so rows with NULL join keys never match each other. If two sources both have NULL customer_id, an inner join drops them all.
- Spark SQL and Databricks have `LEFT SEMI JOIN` and `LEFT ANTI JOIN` as real join types.

PART 1 · FOUNDATIONS · CHAPTER 7

SUBQUERIES

07

A query inside a query. Where it sits decides what it may return: one value, one column, or a whole table.

Kind	Returns	Where it goes	Example
Scalar	One value	SELECT, WHERE	`WHERE amount > (SELECT AVG(amount) FROM orders)`
Column (IN)	One column, many rows	WHERE … IN	`WHERE customer_id IN (SELECT ...)`
Table / derived	Rows and columns	FROM (needs an alias)	`FROM (SELECT ...) AS t`
Correlated	Runs once per outer row	WHERE, SELECT	`WHERE o.amount > (SELECT AVG(...) WHERE x.cust = o.cust)`

Correlated: orders above that customer's own average

THE INNER QUERY SEES O.CUSTOMER_ID — MYSQL 8

```
SELECT o.order_id, o.customer_id, o.amount
FROM orders o
WHERE o.status = 'delivered'
  AND o.amount > (SELECT AVG(x.amount)
                  FROM orders x
                  WHERE x.customer_id = o.customer_id
                    AND x.status = 'delivered')
ORDER BY o.customer_id;
```

RESULT

order_id	customer_id	amount
114	1	2397
111	2	1999
108	5	3498

Customers with a single delivered order never appear: an amount cannot beat its own average. Engines often rewrite this into a join, but in an interview, offer the window version too: compare amount with AVG(amount) OVER (PARTITION BY customer_id).

The NOT IN trap (the most asked SQL trick question)

Question: which customers have never referred anyone? `referred_by` holds the ID of the person who referred them, and it is NULL for most rows.

RETURNS 0 ROWS — MYSQL 8

```
SELECT name
FROM customers
WHERE customer_id NOT IN
      (SELECT referred_by FROM customers);
```

TRAP

`x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`. The last part is UNKNOWN for every x, so the whole AND is never TRUE and every row is dropped.

RETURNS 5 ROWS — MYSQL 8

```
SELECT c.name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM customers r
  WHERE r.referred_by = c.customer_id);
```

RESULT

name
Meera
Rohan
Sana
Vikram
Zoya

	IN / NOT IN	EXISTS / NOT EXISTS	JOIN
NULLs in the list	NOT IN breaks (returns nothing)	Safe	NULL keys never match
Duplicates on the inner side	No effect	No effect, stops at the first match	Multiplies rows
Use it when	Short literal lists	Existence checks, anti joins	You need columns from both tables

INTERVIEW TIP

If you must use NOT IN, filter the NULLs inside: `NOT IN (SELECT referred_by FROM customers WHERE referred_by IS NOT NULL)`. Saying why earns more credit than the query itself.

PART 1 · FOUNDATIONS · CHAPTER 8

CTES AND SET OPERATIONS

08

A CTE names a step so the next step can read it. Set operations stack two result sets on top of each other.

CHAINED CTES READ TOP TO BOTTOM — MYSQL 8

```
WITH delivered AS (            -- step 1: filter
  SELECT * FROM orders WHERE status = 'delivered'
),
monthly AS (                   -- step 2: aggregate
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
         COUNT(*)    AS orders,
         SUM(amount) AS revenue
  FROM delivered
  GROUP BY month
)
SELECT month, orders, revenue,  -- step 3: compare
       revenue > (SELECT AVG(revenue) FROM monthly)
         AS above_avg
FROM monthly
ORDER BY month;
```

RESULT

month	orders	revenue	above_avg
2025-01	2	1994	0
2025-02	2	2847	0
2025-03	4	7291	1
2025-04	4	6840	1

REMEMBER

A later CTE can read any earlier one. The final SELECT can read all of them. MySQL returns 1 / 0 for the boolean column.

	CTE	Subquery	Temp table	View
Lives for	One statement	One statement	The session	Forever (until dropped)
Reusable	Within the query	No	Across queries	Across sessions and users
Stored data	No	No	Yes	No (materialized view: yes)
Reach for it when	Readable multi-step logic	One small filter	Reusing a heavy result 5 times	Sharing logic with a team

Set operations: UNION, UNION ALL, INTERSECT, EXCEPT

Both queries must return the same number of columns with compatible types. Column names come from the first query.

Operator	Returns	Cost
`UNION ALL`	Every row from both, duplicates kept	Cheapest. Default choice when duplicates are impossible or wanted
`UNION`	Rows from both, duplicates removed	Extra sort or hash to dedupe
`INTERSECT`	Rows present in both	MySQL 8.0.31+, all warehouses
`EXCEPT`	Rows in the first, missing from the second	MySQL 8.0.31+. Oracle calls it `MINUS`

EXCEPT: WHAT IS IN A BUT NOT IN B — MYSQL 8

```
-- cities that have customers
SELECT city FROM customers WHERE city IS NOT NULL
EXCEPT
-- ...but no delivered order
SELECT c.city
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered';
```

RESULT

city
Hyderabad

UNION ALL WITH A LABEL COLUMN — MYSQL 8

```
SELECT 'customer' AS type, name
FROM customers WHERE city = 'Delhi'
UNION ALL
SELECT 'employee', name
FROM employees WHERE dept = 'Sales';
```

RESULT

type	name
customer	Kabir
customer	Vikram
employee	Dev
employee	Tara

DATA ENGINEERING NOTE

`EXCEPT` both ways is the fastest data reconciliation check there is: `(A EXCEPT B)` finds rows lost in the load, `(B EXCEPT A)` finds rows that should not be there. Zero rows both ways means the two tables match. Page 36 builds on this.

INTERVIEW TIP

"Is a CTE faster than a subquery?" Usually neither. MySQL 8 and PostgreSQL 12+ fold a CTE that is used once into the main query, just like a subquery. A CTE referenced several times may be computed once and reused, which can help or hurt. PostgreSQL lets you force it with `AS MATERIALIZED` or `AS NOT MATERIALIZED`. Pick CTEs for readability, then check the plan.

PART 1 · FOUNDATIONS · PRACTICE

PRACTICE SET 1: FOUNDATIONS

P1

Cover the solution, write your own, then compare. Each answer runs on the store dataset.

Q4. Total delivered revenue and number of delivered orders per month, newest month first. DA EASY

Approach: Filter delivered, group by a month label, sort descending.

```sql
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       COUNT(*) AS orders, SUM(amount) AS revenue
FROM orders
WHERE status = 'delivered'
GROUP BY month
ORDER BY month DESC;
```

OUTPUT

month	orders	revenue
2025-04	4	6840
2025-03	4	7291
2025-02	2	2847
2025-01	2	1994

Q5. Customers who signed up but have never ordered anything. DA EASY

Approach: Anti join on the primary key of orders.

```sql
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

OUTPUT

customer_id	name
8	Zoya

Q6. For each product category, units sold and revenue from delivered orders. Include categories that sold nothing. DA MEDIUM

Approach: Aggregate delivered lines to the product grain first, then LEFT JOIN from products so every category survives.

```sql
WITH sold AS (
  SELECT i.product_id, SUM(i.qty) AS units,
         SUM(i.qty * i.unit_price) AS revenue
  FROM order_items i
  JOIN orders o ON o.order_id = i.order_id
  WHERE o.status = 'delivered'
  GROUP BY i.product_id
)
SELECT p.category,
       COALESCE(SUM(s.units), 0)   AS units,
       COALESCE(SUM(s.revenue), 0) AS revenue
FROM products p
LEFT JOIN sold s ON s.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

OUTPUT

category	units	revenue
Electronics	10	11790
Home	6	4794
Stationery	12	2388

Filtering status inside a LEFT JOIN chain gets messy fast. Aggregate first, join second.

Q7. Customers whose total delivered spend is above the average customer's total delivered spend. DA MEDIUM

Approach: Two levels of aggregation: per customer, then the average of those totals. A CTE keeps it readable.

```sql
WITH spend AS (
  SELECT customer_id, SUM(amount) AS total
  FROM orders WHERE status = 'delivered'
  GROUP BY customer_id
)
SELECT customer_id, total
FROM spend
WHERE total > (SELECT AVG(total) FROM spend)
ORDER BY total DESC;
```

OUTPUT

customer_id	total
5	5096
1	5094
2	4143

Q8. Employees who earn more than the average salary of their own department. DA DE MEDIUM

Approach: Department average via a derived table joined back on dept.

```sql
SELECT e.name, e.dept, e.salary, d.avg_sal
FROM employees e
JOIN (SELECT dept, ROUND(AVG(salary)) AS avg_sal
      FROM employees GROUP BY dept) d
  ON d.dept = e.dept
WHERE e.salary > d.avg_sal;
```

OUTPUT

name	dept	salary	avg_sal
Anita	Engineering	250000	167500
Rahul	Engineering	180000	167500
Neha	Analytics	150000	135000
Ishita	Analytics	160000	135000

PART 2 · ADVANCED SQL · CHAPTER 9

WINDOW FUNCTIONS 1: RANKING

09

A window function looks at a set of related rows but, unlike GROUP BY, keeps every row. This one idea unlocks half the interview patterns.

```
SUM(amount) OVER ( PARTITION BY customer_id ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW )
```

what to compute | restart per group | order inside group | which rows to look at (frame)

GROUP BY: 9 ROWS BECOME 3 — MYSQL 8

```
SELECT dept, ROUND(AVG(salary)) AS avg_sal
FROM employees
GROUP BY dept;
```

RESULT

dept	avg_sal
Engineering	167500
Analytics	135000
Sales	90000

Every row of a group collapses into one. You lose the names.

WINDOW: 9 ROWS STAY 9 ROWS — MYSQL 8

```
SELECT name, dept, salary,
  ROUND(AVG(salary) OVER (PARTITION BY dept))
    AS dept_avg
FROM employees;
```

RESULT

name	dept	salary	dept_avg
Neha	Analytics	150000	135000
Arjun	Analytics	95000	135000
Ishita	Analytics	160000	135000
Anita	Engineering	250000	167500
Rahul	Engineering	180000	167500
... 4 more rows			

ROW_NUMBER vs RANK vs DENSE_RANK

SAME ORDER, THREE NUMBERING RULES — MYSQL 8

```
SELECT name, salary,
  ROW_NUMBER() OVER (ORDER BY salary DESC, emp_id)
    AS rn,
  RANK()       OVER (ORDER BY salary DESC) AS rnk,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees
ORDER BY salary DESC, emp_id;
```

Function	On a tie	After a tie
ROW_NUMBER	Different numbers	Keeps counting
RANK	Same number	Skips: 1, 2, 2, 4
DENSE_RANK	Same number	No gap: 1, 2, 2, 3

RESULT

name	salary	rn	rnk	drnk
Anita	250000	1	1	1
Rahul	180000	2	2	2
Ishita	160000	3	3	3
Neha	150000	4	4	4
Priya	120000	5	5	5
Karan	120000	6	5	5
Arjun	95000	7	7	6
Dev	90000	8	8	7
Tara	90000	9	8	7

Priya and Karan tie at 1,20,000: RANK gives both 5 and jumps to 7; DENSE_RANK gives both 5 and moves to 6. Dev and Tara tie at the bottom the same way.

APPROACH

Which one to pick? "Top 3 salaries" with ties counting once: DENSE_RANK. "Top 3 people", ties allowed to push past 3: RANK. Exactly one row per group (dedup, latest record): ROW_NUMBER with a tiebreaker.

TRAP

ROW_NUMBER on tied values is not deterministic. Two runs can give Priya and Karan different numbers. Always add a unique column at the end of ORDER BY, like `emp_id`, so results are repeatable.

Also in the family	Returns	Use
NTILE(4) OVER (ORDER BY salary)	Bucket number 1 to 4, roughly equal sizes	Quartiles, deciles, A/B splits
PERCENT_RANK()	(rank - 1) / (rows - 1), from 0 to 1	"Top 10% of customers"
CUME_DIST()	Share of rows with a value at or below this one	Percentile position

INTERVIEW TIP

Windows run after GROUP BY, so you can wrap an aggregate inside one. `SUM(SUM(amount)) OVER ()` is the grand total of the grouped sums, which gives percent of total in a single query: `100 * SUM(amount) / SUM(SUM(amount)) OVER ()`. Pattern P4 on page 22 uses it. What you cannot do is put a window function inside WHERE, GROUP BY or HAVING.

PART 2 · ADVANCED SQL · CHAPTER 10

WINDOW FUNCTIONS 2: FRAMES

10

The frame decides which rows of the partition the function can see. Running totals and moving averages are just frames.

RUNNING TOTAL, 3-ORDER MOVING AVERAGE — MYSQL 8

```
SELECT order_id, order_date, amount,
  SUM(amount) OVER (ORDER BY order_date
     ROWS BETWEEN UNBOUNDED PRECEDING
              AND CURRENT ROW) AS run_total,
  ROUND(AVG(amount) OVER (ORDER BY order_date
     ROWS BETWEEN 2 PRECEDING
              AND CURRENT ROW), 0) AS avg_3
FROM orders
WHERE status = 'delivered'
ORDER BY order_date;
```

RESULT

order_id	order_date	amount	run_total	avg_3
101	2025-01-10	1198	1198	1198
102	2025-01-15	796	1994	997
103	2025-02-02	1499	3493	1164
105	2025-02-18	1348	4841	1214
106	2025-03-01	1048	5889	1298
108	2025-03-12	3498	9387	1965
109	2025-03-20	747	10134	1764
110	2025-03-22	1998	12132	2081
111	2025-04-02	1999	14131	1581
112	2025-04-05	846	14977	1614
114	2025-04-20	2397	17374	1747
115	2025-04-28	1598	18972	1614

REMEMBER

The first two rows of the moving average use only 1 and 2 orders, because nothing came before them. If you need NULL until three rows exist, add COUNT(*) over the same frame and blank out rows where it is below 3.

The default frame trap: ROWS vs RANGE

With ORDER BY and no frame written, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. RANGE treats rows with the same ORDER BY value as one block, so ties get the same running total. ROWS moves one row at a time.

SAME DATA, TWO FRAMES — MYSQL 8

```
SELECT name, salary,
  SUM(salary) OVER (ORDER BY salary)
    AS default_range,
  SUM(salary) OVER (ORDER BY salary, emp_id
    ROWS UNBOUNDED PRECEDING) AS rows_frame
FROM employees
ORDER BY salary, emp_id;
```

RESULT

name	salary	default_range	rows_frame
Dev	90000	180000	90000
Tara	90000	180000	180000
Arjun	95000	275000	275000
Priya	120000	515000	395000
Karan	120000	515000	515000
Neha	150000	665000	665000
Ishita	160000	825000	825000
Rahul	180000	1005000	1005000
Anita	250000	1255000	1255000

Dev and Tara both show 1,80,000 under RANGE; Priya and Karan both show 5,15,000.

INTERVIEW TIP

Write the frame out every time you compute a running total. It costs one line and removes the question. Mention it before they ask: it is a common follow-up.

Frame cheat sheet

Frame	Rows the function sees	Use it for
`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`	Start of partition up to this row	Running total, running count
`ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`	This row and the 6 before it	7-row moving average (needs one row per day)
`RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW`	Rows whose date is within the last 7 days	True 7-day window with missing days (MySQL, PostgreSQL)
`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`	The whole partition	LAST_VALUE, partition totals
`ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`	Previous, current and next row	Centred smoothing
no ORDER BY at all	The whole partition	`SUM(x) OVER (PARTITION BY g)` = group total on every row

PART 2 · ADVANCED SQL · CHAPTER 11

LAG, LEAD AND VALUE FUNCTIONS

11

LAG reads a value from an earlier row, LEAD from a later one. Any question that compares a row with its neighbour lives here.

GAP BETWEEN A CUSTOMER'S ORDERS — MYSQL 8

```
SELECT customer_id AS cust, order_id, order_date,
  LAG(order_date) OVER w AS prev_order,
  DATEDIFF(order_date,
           LAG(order_date) OVER w) AS gap,
  LEAD(order_id) OVER w AS next_id
FROM orders
WHERE customer_id IN (1, 2)
WINDOW w AS (PARTITION BY customer_id
             ORDER BY order_date)
ORDER BY customer_id, order_date;
```

RESULT

cust	order_id	order_date	prev_order	gap	next_id
1	101	2025-01-10	NULL	NULL	103
1	103	2025-02-02	2025-01-10	23	107
1	107	2025-03-05	2025-02-02	31	114
1	114	2025-04-20	2025-03-05	46	NULL
2	102	2025-01-15	NULL	NULL	105
2	105	2025-02-18	2025-01-15	34	111
2	111	2025-04-02	2025-02-18	43	NULL

- The first row in each partition has no previous row, so LAG returns NULL. `LAG(x, 1, 0)` supplies a default instead.
- `LAG(x, 3)` looks three rows back. Handy for "same day last week" on daily data.
- `WINDOW w AS (...)` names a window once and reuses it. Works in MySQL 8, PostgreSQL and BigQuery.

Month over month change in one pass

AGGREGATE FIRST, THEN LAG — MYSQL 8

```
WITH monthly AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
         SUM(amount) AS revenue
  FROM orders WHERE status = 'delivered'
  GROUP BY month
)
SELECT month, revenue,
  revenue - LAG(revenue) OVER (ORDER BY month)
    AS change_abs,
  ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
        / LAG(revenue) OVER (ORDER BY month), 1) AS change_pct
FROM monthly;
```

RESULT

month	revenue	change_abs	change_pct
2025-01	1994	NULL	NULL
2025-02	2847	853	42.8
2025-03	7291	4444	156.1
2025-04	6840	-451	-6.2

LAG on raw orders would compare one order with the previous order. Roll up to the month grain first, then compare months. Pattern P6 on page 23 adds year over year and missing months.

FIRST_VALUE, LAST_VALUE, NTH_VALUE

THE LAST_VALUE TRAP — MYSQL 8

```
SELECT order_id, order_date,
  FIRST_VALUE(order_date) OVER w AS first_order,
  LAST_VALUE(order_date) OVER w AS last_wrong,
  LAST_VALUE(order_date) OVER (w ROWS BETWEEN
      UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
    AS last_right
FROM orders
WHERE customer_id = 1
WINDOW w AS (PARTITION BY customer_id
             ORDER BY order_date);
```

RESULT

order_id	order_date	first_order	last_wrong	last_right
101	2025-01-10	2025-01-10	2025-01-10	2025-04-20
103	2025-02-02	2025-01-10	2025-02-02	2025-04-20
107	2025-03-05	2025-01-10	2025-03-05	2025-04-20
114	2025-04-20	2025-01-10	2025-04-20	2025-04-20

TRAP

LAST_VALUE uses the default frame, which ends at the current row. So the "last" value is always the current row. Widen the frame to UNBOUNDED FOLLOWING, or flip it: FIRST_VALUE with ORDER BY ... DESC.

`NTH_VALUE(order_date, 2) OVER w` returns the second order date once the frame reaches row two. With the default frame it is NULL on row one, so widen the frame here too.

PART 2 · ADVANCED SQL · CHAPTER 12

RECURSIVE CTES

12

A CTE that reads its own output. Use it to walk a hierarchy of unknown depth or to generate rows that do not exist yet, like dates.

Part	What it does	In the org chart below
Anchor member	Runs once. The starting rows.	The boss: `WHERE manager_id IS NULL`
`UNION ALL`	Glues each round onto the result	
Recursive member	Joins the previous round's rows to the table to get the next level	Employees whose `manager_id` is in the last round
Stop condition	Recursion ends when a round returns no rows	Nobody reports to Priya, Karan, Arjun, Ishita or Tara

WALK THE ORG CHART TOP-DOWN — MYSQL 8

```
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 1 AS lvl,
         CAST(name AS CHAR(200)) AS path
  FROM employees
  WHERE manager_id IS NULL            -- anchor
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id,
         o.lvl + 1,
         CONCAT(o.path, ' > ', e.name)
  FROM employees e
  JOIN org o ON e.manager_id = o.emp_id
)                                     -- recurse
SELECT lvl, name, path
FROM org
ORDER BY path;
```

RESULT

lvl	name	path
1	Anita	Anita
2	Dev	Anita > Dev
3	Tara	Anita > Dev > Tara
2	Neha	Anita > Neha
3	Arjun	Anita > Neha > Arjun
3	Ishita	Anita > Neha > Ishita
2	Rahul	Anita > Rahul
3	Karan	Anita > Rahul > Karan
3	Priya	Anita > Rahul > Priya

TRAP

MySQL sizes the path column from the anchor. Without `CAST(... AS CHAR(200))` the column is only as wide as "Anita", and strict mode raises a "data too long" error on the next level.

Generate rows that do not exist: a date spine

ONE ROW PER DAY — MYSQL 8

```
WITH RECURSIVE days AS (
  SELECT DATE('2025-04-01') AS d
  UNION ALL
  SELECT d + INTERVAL 1 DAY
  FROM days
  WHERE d < '2025-04-07'        -- stop
)
SELECT d, DAYNAME(d) AS day_name
FROM days;
```

RESULT

d	day_name
2025-04-01	Tuesday
2025-04-02	Wednesday
2025-04-03	Thursday
2025-04-04	Friday
2025-04-05	Saturday
2025-04-06	Sunday
2025-04-07	Monday

- LEFT JOIN any daily metric to this spine and missing days show up as zero instead of vanishing. That is pattern P18 on page 30.
- PostgreSQL: `generate_series(start, stop, '1 day')`. Snowflake: `GENERATOR` with `ROW_NUMBER`. BigQuery: `GENERATE_DATE_ARRAY`.

DATA ENGINEERING NOTE

MySQL stops runaway recursion at `cte_max_recursion_depth` (1000 by default) and raises an error. PostgreSQL has no such default, so a missing stop condition or a cycle in the data (A manages B, B manages A) runs until you cancel it. For data you do not trust, carry the path and stop when the next id is already in it.

PART 2 · ADVANCED SQL · CHAPTER 13

PIVOT, UNPIVOT AND SUBTOTALS

13

Turning rows into columns is conditional aggregation. Turning columns back into rows is UNION ALL. Subtotals come from ROLLUP.

Pivot: one column per month

CONDITIONAL AGGREGATION PIVOT — MYSQL 8

```
SELECT p.category,
  SUM(CASE WHEN MONTH(o.order_date) = 1
      THEN i.qty * i.unit_price ELSE 0 END) AS jan,
  SUM(CASE WHEN MONTH(o.order_date) = 2
      THEN i.qty * i.unit_price ELSE 0 END) AS feb,
  SUM(CASE WHEN MONTH(o.order_date) = 3
      THEN i.qty * i.unit_price ELSE 0 END) AS mar,
  SUM(CASE WHEN MONTH(o.order_date) = 4
      THEN i.qty * i.unit_price ELSE 0 END) AS apr
FROM order_items i
JOIN orders o   ON o.order_id = i.order_id
JOIN products p ON p.product_id = i.product_id
WHERE o.status = 'delivered'
GROUP BY p.category;
```

RESULT

category	jan	feb	mar	apr
Electronics	799	1499	5096	4396
Home	399	1199	1199	1997
Stationery	796	149	996	447

REMEMBER

You must know the column values in advance. For dynamic columns, build the SQL string in Python or dbt. Snowflake, SQL Server and Databricks also have a `PIVOT` clause; MySQL and PostgreSQL do not (PostgreSQL has `crosstab` in an extension).

Unpivot: columns back to rows

ONE SELECT PER COLUMN — MYSQL 8

```
WITH wide AS (
  SELECT 'Q1' AS qtr, 1994 AS m1,
         2847 AS m2, 7291 AS m3
)
SELECT qtr, 'm1' AS month_no, m1 AS revenue
FROM wide
UNION ALL SELECT qtr, 'm2', m2 FROM wide
UNION ALL SELECT qtr, 'm3', m3 FROM wide;
```

RESULT

qtr	month_no	revenue
Q1	m1	1994
Q1	m2	2847
Q1	m3	7291

Snowflake and Databricks have `UNPIVOT`. In PostgreSQL, a `CROSS JOIN LATERAL (VALUES ...)` does it in one scan.

Subtotals and grand totals with ROLLUP

MYSQL WITH ROLLUP + GROUPING() — MYSQL 8

```
SELECT
  CASE WHEN GROUPING(c.city) = 1
       THEN 'ALL CITIES' ELSE c.city END AS city,
  CASE WHEN GROUPING(o.status) = 1
       THEN 'subtotal' ELSE o.status END AS status,
  SUM(o.amount) AS revenue
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE c.city IN ('Bengaluru', 'Delhi')
GROUP BY c.city, o.status WITH ROLLUP;
```

RESULT

city	status	revenue
Bengaluru	cancelled	1499
Bengaluru	delivered	6142
Bengaluru	returned	798
Bengaluru	subtotal	8439
Delhi	cancelled	1999
Delhi	delivered	1593
Delhi	subtotal	3592
ALL CITIES	subtotal	12031

REMEMBER

GROUPING(col) is 1 on rows ROLLUP added, so you can tell a subtotal from a real NULL. PostgreSQL, Snowflake and BigQuery write `GROUP BY ROLLUP(city, status)` and also support `CUBE` and `GROUPING SETS`.

PART 2 · ADVANCED SQL · PRACTICE

PRACTICE SET 2: ADVANCED

P2

Window functions, recursion and pivots together. Try each one before reading the solution.

Q9. For every employee, show how far their salary is below the top salary in their department. DA EASY

Approach: A partition-level MAX on every row, then subtract. No GROUP BY needed.

```
SELECT name, dept, salary,
  MAX(salary) OVER (PARTITION BY dept) - salary
    AS gap_to_top
FROM employees
ORDER BY dept, gap_to_top;
```

OUTPUT

name	dept	salary	gap_to_top
Ishita	Analytics	160000	0
Neha	Analytics	150000	10000
Arjun	Analytics	95000	65000
Anita	Engineering	250000	0
... 5 more rows			

Q10. Each customer's share of their city's delivered revenue, as a percentage. DA MEDIUM

Approach: Aggregate to customer, then divide by a SUM OVER (PARTITION BY city).

```
WITH spend AS (
  SELECT c.city, c.name, SUM(o.amount) AS total
  FROM orders o
  JOIN customers c ON c.customer_id = o.customer_id
  WHERE o.status = 'delivered' AND c.city IS NOT NULL
  GROUP BY c.city, c.name
)
SELECT city, name, total,
  ROUND(100 * total
    / SUM(total) OVER (PARTITION BY city), 1) AS pct
FROM spend
ORDER BY city, total DESC;
```

OUTPUT

city	name	total	pct
Bengaluru	Aarav	5094	82.9
Bengaluru	Meera	1048	17.1
Delhi	Vikram	846	53.1
Delhi	Kabir	747	46.9
Mumbai	Diya	4143	100.0
Pune	Rohan	5096	100.0

Q11. Everyone who joined through Aarav's referral chain, directly or indirectly, with how many hops away they are. DE MEDIUM

Approach: Recursive CTE on customers.referred_by, anchored on Aarav. Each round finds the people the last round referred.

```
WITH RECURSIVE chain AS (
  SELECT customer_id, name, 0 AS hops
  FROM customers WHERE name = 'Aarav'
  UNION ALL
  SELECT c.customer_id, c.name, ch.hops + 1
  FROM customers c
  JOIN chain ch ON c.referred_by = ch.customer_id
)
SELECT name, hops FROM chain WHERE hops > 0;
```

OUTPUT

name	hops
Diya	1
Meera	2

Q12. Rank customers by delivered spend and show how much more the customer one place above spent. DA HARD

Approach: Aggregate, DENSE_RANK descending, then LAG over the same order to read the row above.

```
WITH spend AS (
  SELECT customer_id, SUM(amount) AS total
  FROM orders WHERE status = 'delivered'
  GROUP BY customer_id
)
SELECT customer_id, total,
  DENSE_RANK() OVER (ORDER BY total DESC) AS rnk,
  LAG(total) OVER (ORDER BY total DESC, customer_id)
    - total AS behind_next
FROM spend
ORDER BY rnk;
```

OUTPUT

customer_id	total	rnk	behind_next
5	5096	1	NULL
1	5094	2	2
2	4143	3	951
6	1998	4	2145
4	1048	5	950
7	846	6	202
3	747	7	99

APPROACH

Before Part 3. Every question on this page was one window function plus one aggregate. The patterns ahead are the same tools arranged in a few repeatable shapes. If any solution here felt shaky, reread pages 14 to 16 first: ranking, frames and LAG carry most of the next ten pages.

PART 3 · INTERVIEW PATTERNS

THE PATTERN MAP

18

Most SQL interview questions are one of these 18 shapes, or two of them stacked. Learn to hear the shape inside the question.

#	Pattern	The question sounds like	Core tool	Pg
P1	**Top-N per group**	*"top 3 products in each category", "highest paid per dept"*	`DENSE_RANK / RANK + filter`	21
P2	**Nth highest value**	*"second highest salary", "3rd largest order"*	`DENSE_RANK or LIMIT ... OFFSET`	21
P3	**Deduplicate, keep one**	*"remove duplicates", "latest record per id"*	`ROW_NUMBER = 1`	22
P4	**Running total, % of total**	*"cumulative revenue", "share of total"*	`SUM() OVER (frame), SUM() OVER ()`	22
P5	**Rolling window**	*"7-day rolling", "trailing 30 days"*	`RANGE INTERVAL frame`	23
P6	**Period over period**	*"MoM growth", "YoY change", "vs last week"*	`LAG, partitioned LAG`	23
P7	**Gaps and islands**	*"consecutive days", "longest streak"*	`date - ROW_NUMBER`	24
P8	**Sessionization**	*"split into sessions", "30 min inactivity"*	`LAG + running SUM of flags`	25
P9	**First / Nth event per entity**	*"first purchase", "time to second order"*	`ROW_NUMBER + conditional MIN`	25
P10	**Cohort retention**	*"retention by first month", "cohort table"*	`first-month CTE + month diff`	26
P11	**Funnel conversion**	*"view to purchase rate", "drop-off by step"*	`per-user step times + order test`	27
P12	**Rates and ratios**	*"cancellation rate", "repeat rate"*	`conditional SUM / COUNT, NULLIF`	27
P13	**Never did X / did all of X**	*"never ordered", "bought every category"*	`anti join, HAVING COUNT DISTINCT`	28
P14	**Row vs related row**	*"earns more than manager", "pairs in a city"*	`self join, a.id < b.id`	28
P15	**Median and percentiles**	*"median order value", "P90 latency"*	`ROW_NUMBER + COUNT, PERCENTILE_CONT`	29
P16	**Overlapping intervals**	*"double bookings", "max concurrent users"*	`a.start < b.end AND b.start < a.end`	29
P17	**Pairs bought together**	*"market basket", "frequently bought with"*	`self join on order, a.p < b.p`	30
P18	**Fill missing dates**	*"show zero days", "every day in the month"*	`date spine + LEFT JOIN`	30

APPROACH

Hearing the pattern in 10 seconds

"each / per / within" means PARTITION BY. "top / highest / latest" means a ranking function. "previous / change / growth" means LAG or LEAD. "consecutive / streak" means islands. "never / without" means an anti join. "every day / including zero" means a spine.

INTERVIEW TIP

Stacked patterns are the HARD questions. "Longest streak of months with growth" is P6 then P7. "Median basket size per cohort" is P10 then P15. Solve the inner pattern in a CTE, check it, then apply the outer one. That is step 05 of the approach on page 4.

One skeleton covers half the list

THE WINDOW-PATTERN SKELETON — ANY ENGINE

```
WITH base AS (          -- 1. filter + fix the grain (aggregate if needed)
  SELECT ... FROM ... WHERE ... GROUP BY ...
),
w AS (                  -- 2. the window step: rank / lag / running sum
  SELECT base.*,
         <fn>() OVER (PARTITION BY <entity> ORDER BY <time or value>) AS x
  FROM base
)
SELECT ...              -- 3. filter or aggregate on the window column
FROM w
WHERE x ...;
```

REMEMBER

How to practise a pattern so it sticks. Read the worked question, close the page, and write the solution from memory on the store data. Then solve one new question with the same pattern on a different table: P7 on order dates instead of logins, P1 on cities instead of categories. If you can do it on a table you have not seen it on, you own the pattern.

P1 TOP-N PER GROUP

DA DE MEDIUM

Sounds like: *"top 3 per region", "best-selling product in each category", "highest paid in each department"*

Worked question: Top 2 products by delivered revenue in each category.

Approach: **1.** aggregate to product grain **2.** rank inside each category **3.** keep rank <= N

SOLUTION — MYSQL 8

```sql
WITH sales AS (
  SELECT p.category, p.product_name,
         SUM(i.qty * i.unit_price) AS revenue
  FROM order_items i
  JOIN orders o   ON o.order_id = i.order_id
  JOIN products p ON p.product_id = i.product_id
  WHERE o.status = 'delivered'
  GROUP BY p.category, p.product_name
),
ranked AS (
  SELECT *, DENSE_RANK() OVER (PARTITION BY category
                  ORDER BY revenue DESC) AS rnk
  FROM sales
)
SELECT category, product_name, revenue, rnk
FROM ranked
WHERE rnk <= 2
ORDER BY category, rnk;
```

RESULT

category	product_name	revenue	rnk
Electronics	Wireless Mouse	4794	1
Electronics	Earbuds	3998	2
Home	Desk Lamp	3597	1
Home	Water Bottle	1197	2
Stationery	Notebook Pack	1494	1
Stationery	Gel Pens	894	2

TRAP

You cannot write WHERE DENSE_RANK() ... <= 2. Windows are computed after WHERE (page 5). Wrap it in a CTE, or use `QUALIFY rnk <= 2` in Snowflake, BigQuery and Databricks.

Pick the ranking function on purpose: DENSE_RANK if tied revenues should both appear, ROW_NUMBER if the business wants exactly N rows. Say which one you chose and why. That sentence is often what the interviewer is listening for.

P2 NTH HIGHEST VALUE

DA DE EASY

Sounds like: *"second highest salary", "third largest order", "Nth highest without LIMIT"*

Worked question: The second highest salary overall, and the second highest in each department.

Approach: **1.** DISTINCT first, so ties do not hide the answer **2.** skip N-1 values **3.** return NULL, not zero rows, when N is too big

TWO SOLUTIONS — MYSQL 8

```sql
-- 2nd highest distinct salary; NULL if it does not exist
SELECT (SELECT DISTINCT salary
        FROM employees
        ORDER BY salary DESC
        LIMIT 1 OFFSET 1) AS second_highest;

-- 2nd highest in every department
SELECT DISTINCT dept, salary
FROM (SELECT dept, salary,
        DENSE_RANK() OVER (PARTITION BY dept
                           ORDER BY salary DESC) AS r
      FROM employees) t
WHERE r = 2;
```

RESULT 1

second_highest
180000

RESULT 2

dept	salary
Analytics	150000
Engineering	180000

REMEMBER

Wrapping the query in an outer SELECT (...) turns "no row" into NULL. That is the detail the classic version of this question checks.

Without windows or LIMIT: `SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees)`.

Sales has no second salary: Dev and Tara tie at 90,000, so DENSE_RANK gives both rank 1 and the department drops out of Result 2. With RANK it would also drop out; with ROW_NUMBER it would wrongly return 90,000.

Follow-ups interviewers ask on P1 and P2

They ask	You answer
"Do it without window functions."	Count how many distinct values beat each row: `WHERE (SELECT COUNT(DISTINCT s2.revenue) FROM sales s2 WHERE s2.category = s.category AND s2.revenue > s.revenue) < 2`. Correct, but it re-scans the group for every row.
"Two products tie for second place."	DENSE_RANK returns both and the group has 3 rows. If they want exactly 2, switch to ROW_NUMBER and name the tiebreaker (product_id, or the newer product).
"The table has a billion rows."	Aggregate before ranking so the window sorts thousands of rows, not billions. In a warehouse, cluster or partition on the PARTITION BY column, and use QUALIFY to skip the extra CTE pass.
"Return the Nth highest as a function."	MySQL: a stored function with `LIMIT 1 OFFSET n-1` (the offset must be a variable set before the query). Mention it, but the scalar subquery above is what they want to see.

P3 DEDUPLICATE, KEEP ONE

DE MEDIUM

Sounds like: *"remove duplicate rows", "keep the latest version of each record", "the CDC feed has repeats"*

Worked question: orders_raw has the same order loaded several times, including one exact duplicate. Keep only the latest row for each order.

Approach: **1.** decide the key that defines "same" **2.** decide which copy wins **3.** ROW_NUMBER, keep 1

SOLUTION — MYSQL 8

```
WITH ranked AS (
  SELECT r.*,
    ROW_NUMBER() OVER (
      PARTITION BY order_id          -- the business key
      ORDER BY ingested_at DESC      -- latest wins
    ) AS rn
  FROM orders_raw r
)
SELECT order_id, status, amount, ingested_at
FROM ranked
WHERE rn = 1
ORDER BY order_id;
```

RESULT

order_id	status	amount	ingested_at
101	delivered	1198	2025-01-12 18:30
102	placed	796	2025-01-15 11:00
103	delivered	1499	2025-02-04 20:00

DATA ENGINEERING NOTE

Snowflake, BigQuery, Databricks: `QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY ingested_at DESC) = 1`.

To delete in place you need something unique per row. Exact duplicates have nothing to tell them apart, so rebuild: CREATE TABLE clean AS the query above, then swap names.

Order 103 had two identical "placed" rows and a later "delivered" row: only the delivered row survives. If two rows tie on ingested_at, add a second ORDER BY column (a load id or file name) so the winner is stable.

P4 RUNNING TOTAL AND % OF TOTAL

DA MEDIUM

Sounds like: *"cumulative revenue by month", "what share of sales does each month make", "which customers make 80%"*

Worked question: Monthly delivered revenue with a running total, each month's share, and the cumulative share.

Approach: **1.** aggregate to month **2.** running SUM with an explicit frame **3.** divide by SUM() OVER () with no ORDER BY

SOLUTION — MYSQL 8

```
WITH monthly AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
         SUM(amount) AS revenue
  FROM orders
  WHERE status = 'delivered'
  GROUP BY month
)
SELECT month, revenue,
  SUM(revenue) OVER (ORDER BY month
      ROWS UNBOUNDED PRECEDING) AS cumulative,
  ROUND(100 * revenue
        / SUM(revenue) OVER (), 1) AS pct,
  ROUND(100 * SUM(revenue) OVER (ORDER BY month
      ROWS UNBOUNDED PRECEDING)
        / SUM(revenue) OVER (), 1) AS cum_pct
FROM monthly
ORDER BY month;
```

RESULT

month	revenue	cumulative	pct	cum_pct
2025-01	1994	1994	10.5	10.5
2025-02	2847	4841	15.0	25.5
2025-03	7291	12132	38.4	63.9
2025-04	6840	18972	36.1	100.0

INTERVIEW TIP

`SUM(x) OVER ()`: empty brackets mean the whole result, so every row sees the grand total. Cumulative share answers "the top customers that make up 80% of revenue": order by revenue DESC and keep rows until cum_pct reaches 80.

P5 ROLLING WINDOW

DA DE MEDIUM

Sounds like: *"7-day rolling average", "trailing 30-day revenue", "sum of the last 3 orders"*

Worked question: For each delivered order, revenue in the 30 days up to and including that date. Compare it with "last 3 orders".

Approach: **1.** decide: N days or N rows? **2.** days: RANGE with INTERVAL **3.** rows: ROWS N PRECEDING

SOLUTION — MYSQL 8

```
SELECT order_date, amount,
  SUM(amount) OVER (ORDER BY order_date
    RANGE BETWEEN INTERVAL 29 DAY PRECEDING
              AND CURRENT ROW) AS rev_30d,
  SUM(amount) OVER (ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING
             AND CURRENT ROW) AS last_3
FROM orders
WHERE status = 'delivered'
ORDER BY order_date;
```

RESULT

order_date	amount	rev_30d	last_3
2025-01-10	1198	1198	1198
2025-01-15	796	1994	1994
2025-02-02	1499	3493	3493
2025-02-18	1348	2847	3643
2025-03-01	1048	3895	3895
2025-03-12	3498	5894	5894
2025-03-20	747	5293	5293
2025-03-22	1998	7291	6243

... 4 more rows

TRAP

ROWS 6 PRECEDING is "the last 7 rows", not "the last 7 days". With missing dates the two differ. Either use RANGE with an INTERVAL, or fill the dates first with a spine (P18) and then use ROWS.

If several orders can land on one day, aggregate to the day first. Engines differ on INTERVAL offsets in RANGE frames; where they are not supported, fill the dates (P18) and use ROWS.

P6 PERIOD OVER PERIOD

DA MEDIUM

Sounds like: *"month over month growth", "year over year", "same day last week"*

Worked question: Year-over-year growth for each month. 2024 values are sample figures typed into the CTE; 2025 values are the store's real months.

Approach: **1.** one row per period first **2.** LAG partitioned by month-of-year, ordered by year **3.** growth = (now - before) / before

SOLUTION — MYSQL 8

```
WITH monthly AS (
  SELECT * FROM (VALUES
    ROW('2024-01-01', 1600), ROW('2024-02-01', 2400),
    ROW('2024-03-01', 5900), ROW('2024-04-01', 7100),
    ROW('2025-01-01', 1994), ROW('2025-02-01', 2847),
    ROW('2025-03-01', 7291), ROW('2025-04-01', 6840)
  ) AS v (month, revenue)
),
yoy AS (
  SELECT month, revenue,
    LAG(revenue) OVER (PARTITION BY MONTH(month)
                       ORDER BY YEAR(month)) AS last_year
  FROM monthly
)
SELECT month, revenue, last_year,
  ROUND(100 * (revenue - last_year) / last_year, 1)
    AS yoy_pct
FROM yoy
ORDER BY month;
```

RESULT

month	revenue	last_year	yoy_pct
2024-01-01	1600	NULL	NULL
2024-02-01	2400	NULL	NULL
2024-03-01	5900	NULL	NULL
2024-04-01	7100	NULL	NULL
2025-01-01	1994	1600	24.6
2025-02-01	2847	2400	18.6
2025-03-01	7291	5900	23.6
2025-04-01	6840	7100	-3.7

TRAP

LAG(revenue, 12) over all months assumes no month is ever missing. One gap and every comparison after it is against the wrong month. Partitioning by month-of-year, or joining on date minus one year, does not have that problem.

P7 GAPS AND ISLANDS

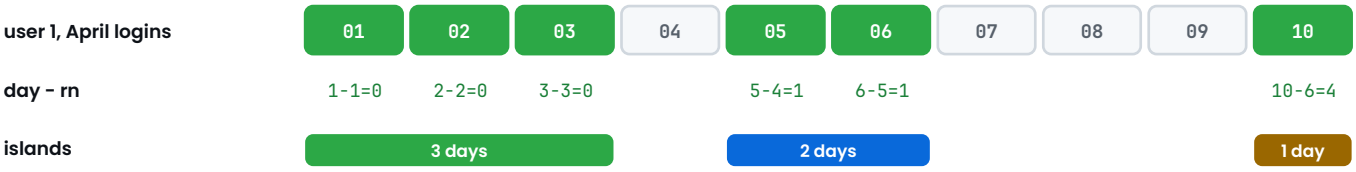
DA DE HARD

Sounds like: *"longest login streak", "consecutive days with sales", "find gaps in invoice numbers", "group back-to-back statuses"*

Worked question: For each user, find every streak of consecutive login days, then report each user's longest streak.

The trick in one picture

Number each user's logins in date order (rn = 1, 2, 3…). Inside a run of consecutive days, the date and rn both go up by 1, so **date minus rn stays constant**. When a day is skipped, the date jumps by 2 but rn only by 1, so the difference changes. That constant is your group id.



Step 1: build the group key

LOOK AT THE INTERMEDIATE RESULT — MYSQL 8

```sql
SELECT user_id, login_date,
  ROW_NUMBER() OVER (PARTITION BY user_id
                     ORDER BY login_date) AS rn,
  DATE_SUB(login_date, INTERVAL ROW_NUMBER()
    OVER (PARTITION BY user_id
          ORDER BY login_date) DAY) AS grp
FROM logins
WHERE user_id = 1;
```

RESULT

user_id	login_date	rn	grp
1	2025-04-01	1	2025-03-31
1	2025-04-02	2	2025-03-31
1	2025-04-03	3	2025-03-31
1	2025-04-05	4	2025-04-01
1	2025-04-06	5	2025-04-01
1	2025-04-10	6	2025-04-04

Three distinct grp values, three islands: 1-3 April, 5-6 April and 10 April. The grp date itself means nothing; only equality matters.

Steps 2 and 3: collapse islands, keep the longest

FULL SOLUTION — MYSQL 8

```sql
WITH g AS (
  SELECT user_id, login_date,
    DATE_SUB(login_date, INTERVAL ROW_NUMBER() OVER (
      PARTITION BY user_id ORDER BY login_date) DAY)
      AS grp
  FROM logins
),
islands AS (                        -- one row per streak
  SELECT user_id, MIN(login_date) AS start_day,
         MAX(login_date) AS end_day, COUNT(*) AS days
  FROM g
  GROUP BY user_id, grp
),
best AS (                           -- longest per user
  SELECT *, RANK() OVER (PARTITION BY user_id
                         ORDER BY days DESC) AS r
  FROM islands
)
SELECT user_id, start_day, end_day, days
FROM best WHERE r = 1
ORDER BY user_id;
```

RESULT

user_id	start_day	end_day	days
1	2025-04-01	2025-04-03	3
2	2025-04-01	2025-04-02	2
3	2025-04-03	2025-04-06	4

TRAP

If a user can log in twice on one day, the dates repeat and the trick breaks. De-duplicate first (`SELECT DISTINCT user_id, login_date`) or use DENSE_RANK instead of ROW_NUMBER.

APPROACH

Same trick, other shapes. Integer ids: `id - ROW_NUMBER()` finds runs of consecutive ids, and the gaps sit between the islands. Repeating statuses: ROW_NUMBER over all rows minus ROW_NUMBER partitioned by status groups back-to-back rows with the same status.

The other half: list the gaps

GAPS SIT BETWEEN ISLANDS — MYSQL 8

```sql
-- reuse g and islands from above, then:
SELECT * FROM (
  SELECT user_id, end_day + INTERVAL 1 DAY AS gap_from,
    LEAD(start_day) OVER (PARTITION BY user_id
      ORDER BY start_day) - INTERVAL 1 DAY AS gap_to
  FROM islands) x
WHERE gap_to IS NOT NULL;
```

RESULT

user_id	gap_from	gap_to
1	2025-04-04	2025-04-04
1	2025-04-07	2025-04-09

LEAD reads the next island's start; the days in between are the gap. The last island has no next one, so its NULL row is dropped.

P8 SESSIONIZATION

DE DA HARD

Sounds like: *"split clickstream into sessions", "a session ends after 30 minutes idle", "average session length"*

Worked question: Break each user's events into sessions, where a gap of more than 30 minutes starts a new session.

Approach: **1.** LAG to get the gap **2.** flag 1 when a new session starts **3.** running SUM of flags = session number

SOLUTION — MYSQL 8

```
WITH flagged AS (
  SELECT user_id, event_time, event_type,
    CASE WHEN TIMESTAMPDIFF(MINUTE,
           LAG(event_time) OVER w, event_time) <= 30
         THEN 0 ELSE 1 END AS new_session
  FROM events
  WINDOW w AS (PARTITION BY user_id
               ORDER BY event_time)
),
numbered AS (
  SELECT *, SUM(new_session) OVER (
    PARTITION BY user_id ORDER BY event_time
    ROWS UNBOUNDED PRECEDING) AS session_no
  FROM flagged
)
SELECT user_id, session_no,
       MIN(event_time) AS started, COUNT(*) AS events
FROM numbered
GROUP BY user_id, session_no
ORDER BY user_id, session_no;
```

RESULT

user_id	session_no	started	events
1	1	2025-04-01 10:00	4
1	2	2025-04-01 14:00	1
2	1	2025-04-01 10:00	2
2	2	2025-04-01 10:40	1
3	1	2025-04-01 11:00	3
4	1	2025-04-01 12:00	1

REMEMBER

The first event has no previous one: TIMESTAMPDIFF returns NULL, NULL <= 30 is UNKNOWN, and CASE falls to ELSE 1. Every user starts at session 1 with no special handling.

User 1 returns at 14:00 and user 2 waits 38 minutes before adding to cart, so both get a second session.

P9 FIRST AND NTH EVENT PER ENTITY

DA MEDIUM

Sounds like: *"first purchase per user", "time between first and second order", "what did they buy first"*

Worked question: For each customer, the first and second order dates and the days between them.

Approach: **1.** number events per entity **2.** pull row 1 and row 2 with conditional MIN **3.** compute the gap

SOLUTION — MYSQL 8

```
WITH seq AS (
  SELECT customer_id, order_date,
    ROW_NUMBER() OVER (PARTITION BY customer_id
                       ORDER BY order_date) AS n
  FROM orders
)
SELECT customer_id,
  MIN(CASE WHEN n = 1 THEN order_date END) AS first,
  MIN(CASE WHEN n = 2 THEN order_date END) AS second,
  DATEDIFF(MIN(CASE WHEN n = 2 THEN order_date END),
           MIN(CASE WHEN n = 1 THEN order_date END))
    AS days_to_2nd
FROM seq
GROUP BY customer_id
ORDER BY customer_id;
```

RESULT

customer_id	first	second	days_to_2nd
1	2025-01-10	2025-02-02	23
2	2025-01-15	2025-02-18	34
3	2025-02-10	2025-03-20	38
4	2025-03-01	2025-04-15	45
5	2025-03-12	2025-04-28	47
6	2025-03-22	NULL	NULL
7	2025-04-05	NULL	NULL

INTERVIEW TIP

Customers with one order get NULL for second, which is correct: they have not come back. "Average days to second order" should exclude them, and AVG already skips NULLs. Say that out loud.

P10 COHORT RETENTION

DA HARD

Sounds like: *"monthly retention by cohort", "what % of January customers came back in month 2", "build a cohort table"*

Worked question: Group customers by the month of their first order. For each cohort, what share ordered again 1, 2 and 3 months later?

Approach: **1.** find each customer's cohort month **2.** month number of every order **3.** distinct customers per (cohort, month) **4.** divide by cohort size and pivot

COHORT TABLE IN FOUR CTE LAYERS — MYSQL 8

```sql
WITH first_order AS (               -- 1. cohort per customer
  SELECT customer_id,
    DATE_FORMAT(MIN(order_date), '%Y-%m-01') AS cohort
  FROM orders
  GROUP BY customer_id
),
activity AS (                       -- 2. month number per order
  SELECT DISTINCT f.customer_id, f.cohort,
    TIMESTAMPDIFF(MONTH, f.cohort,
      DATE_FORMAT(o.order_date, '%Y-%m-01')) AS month_no
  FROM orders o
  JOIN first_order f ON f.customer_id = o.customer_id
),
counts AS (                         -- 3. customers per cell
  SELECT cohort, month_no, COUNT(*) AS active
  FROM activity
  GROUP BY cohort, month_no
)
SELECT DATE_FORMAT(cohort, '%Y-%m') AS cohort,  -- 4. pivot
  MAX(CASE WHEN month_no = 0 THEN active END) AS size,
  ROUND(100 * MAX(CASE WHEN month_no = 1 THEN active END)
    / MAX(CASE WHEN month_no = 0 THEN active END)) AS m1,
  ROUND(100 * MAX(CASE WHEN month_no = 2 THEN active END)
    / MAX(CASE WHEN month_no = 0 THEN active END)) AS m2,
  ROUND(100 * MAX(CASE WHEN month_no = 3 THEN active END)
    / MAX(CASE WHEN month_no = 0 THEN active END)) AS m3
FROM counts
GROUP BY cohort
ORDER BY cohort;
```

RESULT

cohort	size	m1	m2	m3
2025-01	2	100	50	100
2025-02	1	100	NULL	NULL
2025-03	3	67	NULL	NULL
2025-04	1	NULL	NULL	NULL

m1, m2, m3 are % of the cohort still ordering.

REMEMBER

Read across a row: the January cohort had 2 customers; both came back in month 1, one in month 2, both in month 3. NULL means "no data yet" for young cohorts, not zero. Keep that difference when you chart it.

TRAP

Month numbers from `MONTH(b) - MONTH(a)` break across years (January minus December is -11). Use `TIMESTAMPDIFF(MONTH, a, b)` on first-of-month dates, or `DATEDIFF('month', a, b)` in Snowflake.

APPROACH

Interviewers often stop you after step 3, the long table. The pivot is presentation. Offer it, but check the long table first.

Variant	What changes
Cohort by signup, not first order	Use `customers.signup_date` in step 1 and LEFT JOIN orders. Month 0 can then be lower than the cohort size: people who signed up and never bought.
Revenue retention	Replace `COUNT(*)` with `SUM(amount)` in step 3 and drop the DISTINCT in step 2.
Weekly cohorts	Truncate to week (`YEARWEEK` in MySQL, `DATE_TRUNC('week', d)` elsewhere) and count weeks instead of months.
"Unbounded" retention	Active in month N *or any later month*. Take MAX(month_no) per customer and count customers whose max is at least N.

P11 FUNNEL CONVERSION

DA MEDIUM

Sounds like: *"view to purchase conversion", "where do users drop off", "step-by-step funnel"*

Worked question: How many users reached each step of view, cart, checkout, purchase, in that order?

Approach: **1.** one row per user with the first time of each step **2.** a step counts only after the previous one **3.** count per step, then divide

SOLUTION — MYSQL 8

```
WITH steps AS (
  SELECT user_id,
    MIN(CASE WHEN event_type = 'view'
             THEN event_time END) AS t_view,
    MIN(CASE WHEN event_type = 'cart'
             THEN event_time END) AS t_cart,
    MIN(CASE WHEN event_type = 'checkout'
             THEN event_time END) AS t_chk,
    MIN(CASE WHEN event_type = 'purchase'
             THEN event_time END) AS t_buy
  FROM events
  GROUP BY user_id
)
SELECT COUNT(t_view)                     AS viewed,
  SUM(t_cart > t_view)                   AS carted,
  SUM(t_chk > t_cart AND t_cart > t_view) AS checkout,
  SUM(t_buy > t_chk AND t_chk > t_cart
      AND t_cart > t_view)               AS bought
FROM steps;
```

RESULT

viewed	carted	checkout	bought
4	3	2	1

View to purchase: 1 of 4 users, 25%. Each step loses one user.

TRAP

Counting distinct users per event type ignores order: a cart event without a view would still count. Comparing times enforces the sequence. Comparisons with NULL are UNKNOWN and `SUM` skips them, so a missing step counts as 0.

P12 RATES AND RATIOS

DA EASY

Sounds like: *"cancellation rate by city", "repeat purchase rate", "click-through rate"*

Worked question: Cancellation rate per city, with customers who have no city shown as Unknown.

Approach: **1.** numerator with a conditional SUM **2.** denominator with COUNT **3.** guard the division, force decimals

SOLUTION — MYSQL 8

```
SELECT COALESCE(c.city, 'Unknown') AS city,
  COUNT(*) AS orders,
  SUM(o.status = 'cancelled') AS cancelled,
  ROUND(100.0 * SUM(o.status = 'cancelled')
        / NULLIF(COUNT(*), 0), 1) AS cancel_pct
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
GROUP BY COALESCE(c.city, 'Unknown')
ORDER BY cancel_pct DESC, city;
```

RESULT

city	orders	cancelled	cancel_pct
Delhi	3	1	33.3
Bengaluru	6	1	16.7
Mumbai	3	0	0.0
Pune	2	0	0.0
Unknown	1	0	0.0

TRAP

Integer division. In PostgreSQL and SQL Server, `1 / 4` is 0 because both sides are integers. MySQL's `/` returns a decimal, but write `100.0 *` anyway so the query is right on any engine.

`NULLIF(COUNT(*), 0)` turns a zero denominator into NULL instead of an error.

P13 NEVER DID X, DID ALL OF X

DA DE MEDIUM

Sounds like: *"customers who never ordered", "products never sold", "customers who bought from every category"*

Worked question: Which customers have bought from every product category? (The "never" half is the anti join on page 10.)

Approach: **1.** count what each customer covered **2.** count what exists **3.** keep customers where the two match

SOLUTION — MYSQL 8

```sql
SELECT c.name
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id = o.order_id
JOIN products p    ON p.product_id = i.product_id
WHERE o.status = 'delivered'
GROUP BY c.customer_id, c.name
HAVING COUNT(DISTINCT p.category) =
       (SELECT COUNT(DISTINCT category)
        FROM products);
```

RESULT

name
Diya

REMEMBER

This is called **relational division**. Diya bought Stationery, Home and Electronics; everyone else missed at least one.

"Never" version: `WHERE NOT EXISTS (SELECT 1 FROM order_items i WHERE i.product_id = p.product_id)` lists products never sold. Here that is the Phone Stand.

Compare with the categories that exist, not a hard-coded 3. If a category is added tomorrow, the query stays right.

THE NEVER HALF, WRITTEN THREE WAYS — MYSQL 8

```sql
-- customers with no delivered order: three equivalent answers
SELECT c.name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o
                  WHERE o.customer_id = c.customer_id
                    AND o.status = 'delivered');

SELECT c.name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
                  AND o.status = 'delivered'
WHERE o.order_id IS NULL;

SELECT name FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders
                          WHERE status = 'delivered'
                            AND customer_id IS NOT NULL);
```

ALL THREE RETURN

name
Zoya

INTERVIEW TIP

Meera has orders, but one is delivered, so she is not here. Zoya has none at all. Lead with NOT EXISTS: it is NULL-safe and states the intent. Mention the NOT IN guard to show you know the trap from page 11.

P14 ROW VS RELATED ROW

DA DE EASY

Sounds like: *"employees who earn more than their manager", "pairs of users in the same city", "flights with a return leg"*

Worked question: Two classics in one: employees paid more than their manager, and every same-city customer pair.

Approach: **1.** alias the table twice **2.** join on the relationship **3.** compare, or use < to keep each pair once

TWO SELF JOINS — MYSQL 8

```sql
-- employees who earn more than their manager
SELECT e.name AS employee, e.salary,
       m.name AS manager, m.salary AS mgr_salary
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;

-- every pair of customers in the same city, once
SELECT a.city, a.name AS first, b.name AS second
FROM customers a
JOIN customers b ON a.city = b.city
                AND a.customer_id < b.customer_id;
```

RESULT 1

employee	salary	manager	mgr_salary
Ishita	160000	Neha	150000

RESULT 2

city	first	second
Bengaluru	Aarav	Meera
Delhi	Kabir	Vikram

TRAP

`a.customer_id <> b.customer_id` returns every pair twice (Aarav-Meera and Meera-Aarav). `<` keeps one. Tara earns the same as her manager Dev, so "more than" leaves her out. Ask whether "at least as much" was meant.

P15 MEDIAN AND PERCENTILES

DA DE MEDIUM

Sounds like: *"median order value", "P90 delivery time", "median salary per department"*

Worked question: The median delivered order amount, written for MySQL, which has no MEDIAN function.

Approach: **1.** number rows in order and count them **2.** pick the middle row, or the two middle rows **3.** average them

SOLUTION — MYSQL 8

```
WITH ordered AS (
  SELECT amount,
    ROW_NUMBER() OVER (ORDER BY amount) AS rn,
    COUNT(*) OVER () AS cnt
  FROM orders
  WHERE status = 'delivered'
)
SELECT AVG(amount) AS median_amount
FROM ordered
WHERE rn IN (FLOOR((cnt + 1) / 2),
             CEIL((cnt + 1) / 2));
```

RESULT

median_amount
1423.5

12 delivered orders, so rows 6 and 7 are the middle pair: (1348 + 1499) / 2. With an odd count both expressions point at the same row.

REMEMBER

PostgreSQL, Snowflake: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount)`. Snowflake and Databricks also have `MEDIAN(amount)`. For P90 use 0.9. Per group: add `PARTITION BY dept` to both windows and GROUP BY dept.

P16 OVERLAPPING INTERVALS

DE DA HARD

Sounds like: *"double bookings", "overlapping subscriptions", "max concurrent sessions", "merge overlapping ranges"*

Worked question: Find bookings that clash in the same room, then the peak number of bookings live at once.

Approach: **1.** two ranges overlap when each starts before the other ends **2.** self join with that test **3.** for peaks: +1 at start, -1 at end, running sum

CLASHES AND PEAK CONCURRENCY — MYSQL 8

```
-- pairs of bookings that clash in the same room
SELECT a.room_id, a.booking_id AS b1, b.booking_id AS b2
FROM bookings a
JOIN bookings b
  ON  a.room_id = b.room_id
  AND a.booking_id < b.booking_id
  AND a.check_in < b.check_out   -- a starts before b ends
  AND b.check_in < a.check_out;  -- b starts before a ends

-- peak number of bookings live at once, per room
WITH ev AS (
  SELECT room_id, check_in AS t, 1 AS d FROM bookings
  UNION ALL
  SELECT room_id, check_out, -1 FROM bookings
)
SELECT room_id, MAX(live) AS peak
FROM (SELECT room_id, SUM(d) OVER (PARTITION BY room_id
        ORDER BY t, d ROWS UNBOUNDED PRECEDING) AS live
      FROM ev) x
GROUP BY room_id;
```

CLASHES

room_id	b1	b2
10	1	2
20	4	5

PEAK

room_id	peak
10	2
20	2

TRAP

Booking 2 ends on 6 May and booking 3 starts on 6 May. With strict < they do not clash, because checkout frees the room that day. Ordering by d puts the -1 before the +1 at the same moment. Ask whether end dates are inclusive.

Hard follow-up: merge overlapping ranges

"Turn the bookings into continuous occupied periods." This is gaps and islands (P7) on ranges: a booking starts a new island only if it begins after every earlier booking in that room has ended.

MERGE RANGES PER ROOM — MYSQL 8

```
WITH s AS (
  SELECT room_id, booking_id, check_in, check_out,
    MAX(check_out) OVER (PARTITION BY room_id
      ORDER BY check_in, booking_id
      ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS prev_end
  FROM bookings
),
g AS (
  SELECT *, SUM(CASE WHEN prev_end >= check_in THEN 0 ELSE 1 END)
    OVER (PARTITION BY room_id ORDER BY check_in, booking_id
          ROWS UNBOUNDED PRECEDING) AS grp
  FROM s
)
SELECT room_id, MIN(check_in) AS occupied_from,
       MAX(check_out) AS occupied_to, COUNT(*) AS bookings
FROM g
GROUP BY room_id, grp;
```

RESULT

room_id	occupied_from	occupied_to	bookings
10	2025-05-01	2025-05-08	3
20	2025-05-02	2025-05-07	2

The running MAX of earlier end dates, not just the previous row's end, catches a long booking that covers several short ones. Here >= also merges back-to-back stays into one period.

P17 PAIRS BOUGHT TOGETHER

DA MEDIUM

Sounds like: *"frequently bought together", "market basket", "which two products co-occur most"*

Worked question: Which pairs of products appear together in delivered orders most often?

Approach: **1.** self join order lines on the same order **2.** a.product_id < b.product_id keeps each pair once **3.** count orders per pair

SOLUTION — MYSQL 8

```sql
SELECT pa.product_name AS product_a,
       pb.product_name AS product_b,
       COUNT(*) AS together
FROM order_items a
JOIN order_items b ON b.order_id = a.order_id
                  AND a.product_id < b.product_id
JOIN orders o    ON o.order_id = a.order_id
JOIN products pa ON pa.product_id = a.product_id
JOIN products pb ON pb.product_id = b.product_id
WHERE o.status = 'delivered'
GROUP BY pa.product_name, pb.product_name
ORDER BY together DESC, product_a, product_b
LIMIT 5;
```

RESULT

product_a	product_b	together
Wireless Mouse	Desk Lamp	2
Wireless Mouse	Water Bottle	2
Desk Lamp	Gel Pens	1
Desk Lamp	Water Bottle	1
Notebook Pack	Gel Pens	1

REMEMBER

The Wireless Mouse sits in both top pairs: it shares two orders with the Desk Lamp and two with the Water Bottle.

A real follow-up adds support (pair orders / all orders) and lift. The self join is the part they test.

DATA ENGINEERING NOTE

An order with k lines makes k(k-1)/2 pairs. Big baskets explode, so filter rows before the self join, not after.

P18 FILL MISSING DATES

DA DE MEDIUM

Sounds like: *"daily revenue including days with no sales", "show every date in the range", "zero-fill a time series"*

Worked question: Delivered revenue for every day from 1 to 7 April, zeros included, plus a true 3-day rolling sum.

Approach: **1.** generate the spine **2.** aggregate facts to the same grain **3.** LEFT JOIN facts onto the spine, COALESCE to 0

SOLUTION — MYSQL 8

```sql
WITH RECURSIVE days AS (
  SELECT DATE('2025-04-01') AS d
  UNION ALL
  SELECT d + INTERVAL 1 DAY FROM days
  WHERE d < '2025-04-07'
),
daily AS (
  SELECT order_date, SUM(amount) AS revenue
  FROM orders
  WHERE status = 'delivered'
  GROUP BY order_date
)
SELECT days.d AS day,
  COALESCE(daily.revenue, 0) AS revenue,
  SUM(COALESCE(daily.revenue, 0)) OVER (
    ORDER BY days.d ROWS BETWEEN 2 PRECEDING
                            AND CURRENT ROW) AS rev_3day
FROM days
LEFT JOIN daily ON daily.order_date = days.d
ORDER BY day;
```

RESULT

day	revenue	rev_3day
2025-04-01	0	0
2025-04-02	1999	1999
2025-04-03	0	1999
2025-04-04	0	1999
2025-04-05	846	846
2025-04-06	0	846
2025-04-07	0	846

APPROACH

Start FROM the spine and LEFT JOIN the data onto it. Starting FROM the data loses the empty days before they can be filled.

Once every day has a row, ROWS 2 PRECEDING really is "the last 3 days". P5 and P18 work as a pair.

DATA ENGINEERING NOTE

Warehouses keep a permanent date dimension (dim_date) for this. Join to it instead of generating dates in every query.

PART 4 · SQL FOR DATA ENGINEERING · CHAPTER 14

DDL, KEYS AND MODELLING

14

DE rounds move from "query this table" to "design this table". Constraints, keys and the shape of the schema come first.

EVERY CONSTRAINT IN ONE TABLE — MYSQL 8

```
CREATE TABLE payments (
  payment_id  BIGINT AUTO_INCREMENT PRIMARY KEY, -- surrogate
  order_id    INT NOT NULL,
  paid_at     DATETIME NOT NULL
              DEFAULT CURRENT_TIMESTAMP,
  method      VARCHAR(10) NOT NULL,
  amount      DECIMAL(10,2) NOT NULL,
  gateway_ref VARCHAR(40) UNIQUE,                -- natural key
  CONSTRAINT fk_pay_order FOREIGN KEY (order_id)
    REFERENCES orders (order_id),
  CONSTRAINT chk_amount CHECK (amount > 0),
  CONSTRAINT chk_method CHECK (method IN ('upi','card','cod'))
);
```

- `NOT NULL` stops missing values at the door instead of in a report.
- `CHECK` enforces business rules: no zero payments, only known methods (MySQL 8.0.16+ enforces it; older versions ignored it).
- `FOREIGN KEY` blocks a payment for an order that does not exist.
- `UNIQUE` on gateway_ref stops the same gateway callback being saved twice.

Key	Means
Primary key	Unique + NOT NULL. One per table. Identifies the row.
Candidate key	Any column set that could be the PK (gateway_ref here).
Unique key	No duplicates, but NULLs allowed (one or many, by engine).
Foreign key	Must match a PK in the parent table. Blocks orphans.
Composite key	PK over several columns: (order_id, product_id).
Surrogate vs natural	Surrogate: meaningless id you generate. Natural: business id (PAN, email). Warehouses join on surrogates so history (SCD 2) can have many rows per natural key.

Warehouses such as Snowflake, BigQuery and Databricks accept PRIMARY KEY and FOREIGN KEY but **do not enforce them** (NOT NULL is enforced). The DQ checks on page 36 are how you enforce them there.

Normal forms in one table

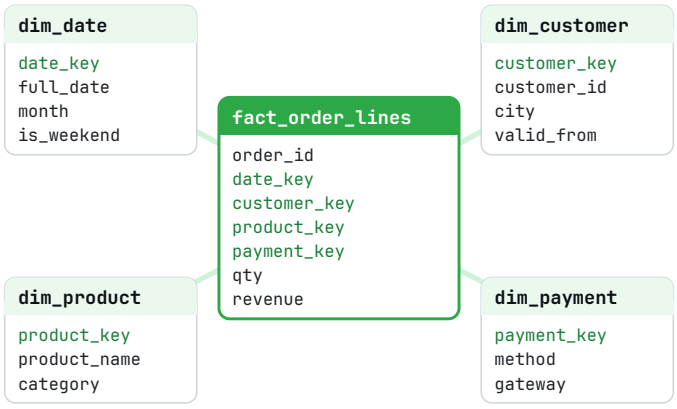
Form	Rule	Breaks it	Fix
1NF	One value per cell, no repeating groups	`orders.items = 'Mouse, Bottle'`	One row per item: order_items
2NF	1NF + every column depends on the whole composite key	`order_items(order_id, product_id, product_name)`	product_name depends on product_id only: move it to products
3NF	2NF + no column depends on another non-key column	`customers(customer_id, city, state)`	state depends on city: move to a cities table

OLTP vs OLAP

	OLTP (MySQL, PostgreSQL)	OLAP (Snowflake, BigQuery)
Job	Run the app: place an order	Answer questions over history
Schema	Normalized (3NF)	Denormalized star schema
Queries	Many tiny reads and writes by key	Few huge scans and aggregates
Storage	Row oriented	Column oriented, compressed

	DELETE	TRUNCATE	DROP
Removes	Chosen rows	All rows	Table + structure
WHERE	Yes	No	No
Rollback	Yes	MySQL: no (DDL)	No
Speed	Row by row	Fast, resets AUTO_INCREMENT	Fast

A star schema



The fact holds numbers at one grain (one row per order line). Dimensions hold the descriptive columns you filter and group by. A snowflake schema normalizes the dimensions further (product to category table).

PART 4 · SQL FOR DATA ENGINEERING · CHAPTER 15

INDEXES AND QUERY PERFORMANCE 15

An index is a sorted copy of some columns that lets the engine jump to rows instead of reading all of them. EXPLAIN shows whether it did.

ONE INDEX, THREE QUERIES — MYSQL 8

```
CREATE INDEX idx_orders_cust_date
  ON orders (customer_id, order_date);

-- A: both columns, left to right
EXPLAIN SELECT order_id, amount FROM orders
WHERE customer_id = 1
  AND order_date >= '2025-03-01';

-- B: skips the first column
EXPLAIN SELECT order_id FROM orders
WHERE order_date >= '2025-03-01';

-- C: function wrapped around the column
EXPLAIN SELECT order_id, amount FROM orders
WHERE YEAR(order_date) = 2025;
```

REAL EXPLAIN OUTPUT, MYSQL 8.0

	type	key	rows	Extra
A	range	idx_orders_cust_date	2	Using index condition
B	index	idx_orders_cust_date	15	Using where; Using index
C	ALL	NULL	15	Using where

type	Means
const / eq_ref / ref	Index lookup on equality. Best.
range	Index seek on a range. Good.
index	Reads the whole index. Better than ALL only because the index is smaller.
ALL	Full table scan. Fine on 15 rows, a problem on 15 crore.

A uses the index as designed. B cannot seek because the index is sorted by customer_id first (the leftmost prefix rule); it scans the whole index. C hides order_date inside YEAR(), so no index on it can help.

Sargable rewrites: let the engine use the index

Slow (no index seek)	Fast (sargable)	Why
`WHERE YEAR(order_date) = 2025`	`WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'`	Function on the column hides it from the index
`WHERE amount * 1.18 > 1000`	`WHERE amount > 1000 / 1.18`	Do the math on the constant side
`WHERE name LIKE '%ra%'`	`WHERE name LIKE 'Ra%'`	A leading wildcard cannot use a B-tree; use full-text search if you need it
`WHERE phone = 9876543210`	`WHERE phone = '9876543210'`	Type mismatch forces a conversion on every row
`WHERE city = 'Pune' OR customer_id = 7`	Two queries joined with UNION	OR across different columns often defeats both indexes

REMEMBER

Index trade-offs to say out loud. Every index speeds reads and slows writes: each INSERT and UPDATE maintains it. A **covering index** holds every column the query needs, so the table is never touched ("Using index" in Extra). Composite index order: equality columns first, then the range column.

DATA ENGINEERING NOTE

Warehouses mostly do not use B-tree indexes. Speed comes from reading less: select only the columns you need (column storage), filter on the partition column so whole files are skipped (pruning), cluster on common filters, pre-aggregate, and broadcast the small side of a join. Check the query profile the way you would read EXPLAIN.

PART 4 · SQL FOR DATA ENGINEERING · CHAPTER 16

TRANSACTIONS AND ACID

16

A transaction groups statements so they succeed or fail together. Isolation levels decide what concurrent transactions can see of each other.

Letter	Guarantee	Example in the store
Atomicity	All statements commit, or none do	Order row and its items are written together, never half
Consistency	Constraints hold before and after	CHECK (amount > 0) and foreign keys are never violated at commit
Isolation	Concurrent transactions do not see each other's half-done work	A report never sees an order without its items
Durability	Committed means it survives a crash	Written to the redo log before COMMIT returns

COMMIT, SAVEPOINT, ROLLBACK — MYSQL 8

```
START TRANSACTION;
UPDATE wallets SET balance = balance - 300 WHERE user_id = 1;
UPDATE wallets SET balance = balance + 300 WHERE user_id = 2;
COMMIT;

-- a transfer that must fail as a whole
START TRANSACTION;
UPDATE wallets SET balance = balance + 900 WHERE user_id = 2;
SAVEPOINT credited;
UPDATE wallets SET balance = balance - 900 WHERE user_id = 1;
-- CHECK fails: 700 - 900 < 0. The app catches the error:
ROLLBACK;   -- undoes the credit too (ROLLBACK TO credited
            -- would undo only the steps after the savepoint)
```

BALANCES AFTER BOTH TRANSACTIONS

user_id	balance
1	700
2	800

TRAP

Autocommit is on by default in MySQL and PostgreSQL: each statement commits on its own. Without START TRANSACTION, a crash between the two UPDATEs loses 300 rupees. DDL like CREATE or TRUNCATE commits implicitly in MySQL, even inside a transaction.

Isolation levels and the anomalies they allow

Level	Dirty read	Non-repeatable read	Phantom read	Default in
READ UNCOMMITTED	possible	possible	possible	rarely used
READ COMMITTED	prevented	possible	possible	PostgreSQL, SQL Server, Oracle, Snowflake
REPEATABLE READ	prevented	prevented	possible by the standard (InnoDB blocks most with snapshots and gap locks)	MySQL InnoDB
SERIALIZABLE	prevented	prevented	prevented	opt-in everywhere

Dirty read: you see another transaction's uncommitted change. **Non-repeatable read:** the same row read twice gives different values. **Phantom:** the same WHERE read twice returns new rows.

The lost update

Time	Session A	Session B
t1	reads stock = 5	
t2		reads stock = 5
t3	writes stock = 4	
t4		writes stock = 4 (one sale lost)

INTERVIEW TIP

Deadlock: A locks row 1 then wants row 2; B locks row 2 then wants row 1. The engine kills one. Prevent it by locking rows in the same order everywhere, keeping transactions short, and retrying the victim.

TWO FIXES — MYSQL 8

```
-- assume products had a stock column
-- fix 1: let the database do the arithmetic
UPDATE products SET stock = stock - 1
WHERE product_id = 4 AND stock > 0;

-- fix 2: lock the row while you decide
START TRANSACTION;
SELECT stock FROM products
WHERE product_id = 4 FOR UPDATE;
-- ... checks in app code ...
UPDATE products SET stock = 4 WHERE product_id = 4;
COMMIT;
```

PART 4 · SQL FOR DATA ENGINEERING · CHAPTER 17

INCREMENTAL LOADS AND UPSERTS

17

Reloading everything every night stops working as tables grow. Load only what changed, and make the load safe to run twice.

Strategy	How	Good for	Watch out for
Full refresh	TRUNCATE + INSERT everything	Small dimensions	Cost grows with history
Incremental append	Insert rows with `created_at > watermark`	Immutable events, logs	Late events, duplicates
Incremental upsert	Rows with `updated_at > watermark`, MERGE on key	Mutable entities: orders	Deletes are invisible
Partition overwrite	DELETE a day, re-INSERT that day	Daily facts, backfills	Must be one transaction
CDC	Read the database log (Debezium, DMS)	Deletes, high volume	Ordering, schema changes

WATERMARK + UPSERT, MYSQL — MYSQL 8

```
START TRANSACTION;

INSERT INTO dw_orders
SELECT * FROM (
  SELECT order_id, customer_id, order_date, status,
         amount, updated_at
  FROM src_orders
  WHERE updated_at > (SELECT last_loaded_at - INTERVAL 1 DAY
                      FROM etl_watermark
                      WHERE table_name = 'orders')
) AS new_rows
ON DUPLICATE KEY UPDATE
  status     = new_rows.status,
  amount     = new_rows.amount,
  updated_at = new_rows.updated_at;

UPDATE etl_watermark
SET last_loaded_at = (SELECT MAX(updated_at) FROM dw_orders)
WHERE table_name = 'orders';

COMMIT;
```

Setup: the warehouse holds orders up to 22 March. Since then, five April orders arrived and order 110 was changed to returned.

AFTER THE LOAD

dw_rows	order_110	new_watermark
15	returned	2025-04-28 09:00

APPROACH

Why the 1-day lookback? Rows can commit in the source after you read the watermark, with an older updated_at. Re-reading a small overlap catches them, and the upsert makes the overlap harmless. That is what idempotent means here: running the load twice gives the same table.

THE SAME UPSERT AS MERGE — SNOWFLAKE · DATABRICKS · PG 15+

```
MERGE INTO dw_orders AS t
USING stg_orders AS s
  ON t.order_id = s.order_id
WHEN MATCHED AND s.updated_at > t.updated_at THEN
  UPDATE SET status = s.status, amount = s.amount,
             updated_at = s.updated_at
WHEN NOT MATCHED THEN
  INSERT (order_id, customer_id, order_date,
          status, amount, updated_at)
  VALUES (s.order_id, s.customer_id, s.order_date,
          s.status, s.amount, s.updated_at);
```

- `WHEN MATCHED AND s.updated_at > t.updated_at` stops an older record from overwriting a newer one if files arrive out of order.
- Databricks and Snowflake also take `WHEN NOT MATCHED BY SOURCE THEN DELETE` (Databricks) or a soft-delete flag column, which is how a MERGE handles deletes.
- The source must have one row per key. Two staging rows for the same order_id make MERGE fail (Snowflake, Databricks), or insert the key twice when it is new. Deduplicate staging first: that is P3.

PART 4 · SQL FOR DATA ENGINEERING · CHAPTER 18

SLOWLY CHANGING DIMENSIONS

18

When a customer moves city, do you overwrite the old city or keep history? Type 1 overwrites. Type 2 keeps a new row per version.

Type	On change	History	Use when
SCD 0	Never change	Original value only	Signup date, date of birth
SCD 1	UPDATE in place	Lost	Typo fixes, values nobody reports on historically
SCD 2	Close the old row, insert a new one	Full, row per version	City, segment, price: anything reports must see "as it was"
SCD 3	Keep a previous_city column	One step back	Rare: "current vs previous" only

SCD 2 IN TWO STATEMENTS, MYSQL — MYSQL 8

```
START TRANSACTION;

-- 1. close the current row when the city changed
UPDATE dim_customer d
JOIN stg_customer s ON s.customer_id = d.customer_id
SET d.valid_to = s.load_date - INTERVAL 1 DAY,
    d.is_current = 0
WHERE d.is_current = 1
  AND NOT (d.city <=> s.city);      -- NULL-safe compare

-- 2. insert a new current row for new or changed keys
INSERT INTO dim_customer
  (customer_id, name, city, valid_from, valid_to, is_current)
SELECT s.customer_id, s.name, s.city,
       s.load_date, '9999-12-31', 1
FROM stg_customer s
LEFT JOIN dim_customer d
  ON d.customer_id = s.customer_id AND d.is_current = 1
WHERE d.customer_id IS NULL;        -- no open row left

COMMIT;
```

Incoming: Diya unchanged (Mumbai), Meera moved Bengaluru to Pune, Sana's city went from NULL to Chennai.

DIM_CUSTOMER AFTER THE LOAD

sk	id	city	valid_from	valid_to	cur
2	2	Mumbai	2025-01-12	9999-12-31	1
4	4	Bengaluru	2025-02-20	2025-04-30	0
16	4	Pune	2025-05-01	9999-12-31	1
6	6	NULL	2025-03-15	2025-04-30	0
17	6	Chennai	2025-05-01	9999-12-31	1

TRAP

With `d.city <> s.city`, Sana's change is missed: NULL <> 'Chennai' is UNKNOWN. Use `<=>` in MySQL or `IS DISTINCT FROM` elsewhere. With many tracked columns, compare a hash of them instead.

POINT-IN-TIME JOIN — MYSQL 8

```
SELECT o.order_id, o.order_date,
       d_then.city AS city_at_order,
       d_now.city  AS city_today
FROM store.orders o
JOIN dim_customer d_then
  ON d_then.customer_id = o.customer_id
 AND o.order_date BETWEEN d_then.valid_from
                      AND d_then.valid_to
JOIN dim_customer d_now
  ON d_now.customer_id = o.customer_id
 AND d_now.is_current = 1
WHERE o.customer_id = 4;
```

RESULT

order_id	order_date	city_at_order	city_today
106	2025-03-01	Bengaluru	Pune
113	2025-04-15	Bengaluru	Pune

Revenue reports by city must use city_at_order, or Meera's March and April orders move to Pune after the fact. Many teams store valid_to as the next valid_from and filter with >= and < instead of BETWEEN; both work if you are consistent.

DATA ENGINEERING NOTE

In Snowflake and Databricks the same SCD 2 load is usually one `MERGE`: the staging rows are unioned with a copy that has a NULL merge key, so changed customers match once (to close the old row) and also fail to match once (to insert the new row). dbt snapshots generate this for you.

PART 4 · SQL FOR DATA ENGINEERING · CHAPTER 19

DATA QUALITY IN SQL

19

A pipeline that finishes green can still load wrong data. These checks catch it, and every one of them is a plain query.

ONE QUERY, ONE ROW PER CHECK — MYSQL 8

```sql
SELECT 'duplicate order_id' AS check_name,
       COUNT(*) - COUNT(DISTINCT order_id) AS failed
FROM orders
UNION ALL
SELECT 'orphan customer_id', COUNT(*)
FROM orders o LEFT JOIN customers c
  ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL
UNION ALL
SELECT 'amount not positive', COUNT(*)
FROM orders WHERE amount <= 0 OR amount IS NULL
UNION ALL
SELECT 'status outside allowed list', COUNT(*)
FROM orders
WHERE status NOT IN ('delivered', 'cancelled', 'returned')
UNION ALL
SELECT 'order total <> sum of items', COUNT(*)
FROM orders o
JOIN (SELECT order_id, SUM(qty * unit_price) AS s
      FROM order_items GROUP BY order_id) i
  ON i.order_id = o.order_id
WHERE i.s <> o.amount
UNION ALL
SELECT 'customer city missing', COUNT(*)
FROM customers WHERE city IS NULL
UNION ALL
SELECT 'exact duplicate raw rows',
  COUNT(*) - COUNT(DISTINCT order_id, status,
                           amount, ingested_at)
FROM orders_raw;
```

RESULT

check_name	failed
duplicate order_id	0
orphan customer_id	0
amount not positive	0
status outside allowed list	0
order total <> sum of items	0
customer city missing	1
exact duplicate raw rows	1

Two checks fail, both planted: Sana's missing city and the doubled raw row for order 103. In a pipeline, run this after each load and stop the job (or alert) when a blocking check is above zero.

INTERVIEW TIP

Freshness is a check too: `TIMESTAMPDIFF(HOUR, MAX(loaded_at), NOW())` above the SLA means the data is stale even though every other check passes.

Reconcile source and target

ZERO ROWS BACK MEANS THEY MATCH — ANY ENGINE WITH EXCEPT

```sql
-- 1. counts and sums per day must match
SELECT order_date, COUNT(*) AS n, SUM(amount) AS amt
FROM source_orders GROUP BY order_date
EXCEPT
SELECT order_date, COUNT(*), SUM(amount)
FROM dw_orders GROUP BY order_date;

-- 2. row level, both directions
SELECT 'missing in dw' AS issue, s.* FROM
  (SELECT * FROM source_orders
   EXCEPT SELECT * FROM dw_orders) s
UNION ALL
SELECT 'extra in dw', d.* FROM
  (SELECT * FROM dw_orders
   EXCEPT SELECT * FROM source_orders) d;
```

Check	Catches
Row count by day	Dropped or duplicated loads
Sum of amounts by day	Truncated decimals, wrong joins
Row-level EXCEPT	Exactly which rows differ
Distinct keys	Fan-out from a bad join
NULL rate per column	A source column silently went empty

DATA ENGINEERING NOTE

Tools wrap these same queries: dbt tests (unique, not_null, relationships, accepted_values), Great Expectations, Soda, and Delta Live Tables expectations. Knowing the SQL underneath is what interviews check.

Warehouse feature	Snowflake	BigQuery	Databricks (Delta)
Skip data on filters	Micro-partitions + clustering key	PARTITION BY date + CLUSTER BY	Partitioning, Z-ORDER, liquid clustering
Filter a window result	`QUALIFY`	`QUALIFY`	`QUALIFY`
Upsert	`MERGE`	`MERGE`	`MERGE INTO`
Stored, auto-refreshed result	Materialized view, dynamic table	Materialized view	Materialized view, streaming table
Query old versions	`AT (TIMESTAMP => ...)`	`FOR SYSTEM_TIME AS OF`	`VERSION AS OF`

PART 5 · PRACTICE AND REVISION

INTERVIEW SET: DATA ANALYTICS

DA

Mixed questions in the style of DA SQL rounds. Name the pattern before you write anything; it is printed under each solution.

Q31. The top customer by delivered spend in each month. Keep ties. DA MEDIUM

Approach: Aggregate to (month, customer), RANK within month, keep rank 1. Pattern P1.

```sql
WITH m AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
         customer_id, SUM(amount) AS spend
  FROM orders WHERE status = 'delivered'
  GROUP BY month, customer_id
)
SELECT month, customer_id, spend
FROM (SELECT *, RANK() OVER (PARTITION BY month
             ORDER BY spend DESC) AS rk FROM m) r
WHERE rk = 1 ORDER BY month;
```

OUTPUT

month	customer_id	spend
2025-01	1	1198
2025-02	1	1499
2025-03	5	3498
2025-04	1	2397

Q32. What percentage of customers placed a second order within 45 days of their first? DA MEDIUM

Approach: First and second order per customer (P9), then a conditional ratio over all customers who ordered (P12).

```sql
WITH seq AS (
  SELECT customer_id, order_date,
    ROW_NUMBER() OVER (PARTITION BY customer_id
                       ORDER BY order_date) AS n
  FROM orders
), f AS (
  SELECT customer_id,
    MIN(CASE WHEN n = 1 THEN order_date END) AS d1,
    MIN(CASE WHEN n = 2 THEN order_date END) AS d2
  FROM seq GROUP BY customer_id
)
SELECT COUNT(*) AS customers,
  SUM(DATEDIFF(d2, d1) <= 45) AS repeat_45d,
  ROUND(100.0 * SUM(DATEDIFF(d2, d1) <= 45)
        / COUNT(*), 1) AS pct
FROM f;
```

OUTPUT

customers	repeat_45d	pct
7	4	57.1

The denominator is every customer who ordered, including one-order customers. State that choice; some teams use signups instead.

Q33. The longest run of consecutive months in which delivered revenue grew over the previous month. DA HARD

Approach: Monthly revenue, a growth flag with LAG (P6), then islands on the flag (P7): row number minus row number within the flag.

```sql
WITH m AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m-01') AS mon,
         SUM(amount) AS rev
  FROM orders WHERE status = 'delivered' GROUP BY mon
), g AS (
  SELECT mon,
    COALESCE(rev > LAG(rev) OVER (ORDER BY mon), 0) AS grew
  FROM m
), isl AS (
  SELECT mon, grew,
    ROW_NUMBER() OVER (ORDER BY mon)
    - ROW_NUMBER() OVER (PARTITION BY grew ORDER BY mon) AS grp
  FROM g
)
SELECT MIN(mon) AS from_month, MAX(mon) AS to_month,
       COUNT(*) AS months
FROM isl WHERE grew = 1
GROUP BY grp ORDER BY months DESC LIMIT 1;
```

OUTPUT

from_month	to_month	months
2025-02-01	2025-03-01	2

Assumes every month has sales. If one could be empty, build the months from a spine first (P18).

Q34. Customers whose every order was delivered (no cancellations or returns). DA EASY

Approach: Group per customer and require zero non-delivered rows. A "did all of X" question (P13) without a second table.

```sql
SELECT c.name, COUNT(*) AS orders
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
HAVING SUM(o.status <> 'delivered') = 0
ORDER BY c.name;
```

OUTPUT

name	orders
Diya	3
Rohan	2
Sana	1
Vikram	1

PART 5 · PRACTICE AND REVISION

INTERVIEW SET: DATA ENGINEERING

DE

DE rounds test the same SQL on messier data: change logs, gaps, late rows and two systems that disagree.

Q35. A source sends sequential ids. Given ids 1, 2, 3, 5, 6, 9, list every missing range. DE MEDIUM

Approach: LEAD gives the next id. Any jump bigger than 1 is a gap from id + 1 to next_id - 1.

```
WITH ids AS (
  SELECT * FROM (VALUES ROW(1), ROW(2), ROW(3),
                ROW(5), ROW(6), ROW(9)) AS v (id)
), n AS (
  SELECT id, LEAD(id) OVER (ORDER BY id) AS next_id
  FROM ids
)
SELECT id + 1 AS gap_start, next_id - 1 AS gap_end
FROM n
WHERE next_id - id > 1;
```

OUTPUT

gap_start	gap_end
4	4
7	8

Q36. From the change log orders_raw, what was the status of each order as of 2025-02-03 00:00? DE MEDIUM

Approach: Point in time: keep only rows ingested before the cut-off, then the latest per key (P3).

```
SELECT order_id AS id, status, ingested_at
FROM (
  SELECT r.*, ROW_NUMBER() OVER (PARTITION BY order_id
           ORDER BY ingested_at DESC) AS rn
  FROM orders_raw r
  WHERE ingested_at < '2025-02-03 00:00:00'
) x
WHERE rn = 1;
```

OUTPUT

id	status	ingested_at
101	delivered	2025-01-12 18:30
102	placed	2025-01-15 11:00
103	placed	2025-02-02 08:15

Order 103 was still "placed" at the cut-off; it became delivered on 4 Feb.

Q37. Turn the change log into validity ranges (valid_from, valid_to) for each order status. DE HARD

Approach: Remove exact duplicates, then LEAD(ingested_at) is when the current version stopped being true. Open versions get 9999-12-31.

```
WITH dedup AS (
  SELECT DISTINCT order_id, status, ingested_at
  FROM orders_raw
)
SELECT order_id AS id, status,
  ingested_at AS valid_from,
  COALESCE(LEAD(ingested_at) OVER (PARTITION BY order_id
             ORDER BY ingested_at),
           '9999-12-31 00:00:00') AS valid_to
FROM dedup
ORDER BY id, valid_from;
```

OUTPUT

id	status	valid_from	valid_to
101	placed	2025-01-10 09:00	2025-01-12 18:30:00
101	delivered	2025-01-12 18:30	9999-12-31 00:00:00
102	placed	2025-01-15 11:00	9999-12-31 00:00:00
103	placed	2025-02-02 08:15	2025-02-04 20:00:00
103	delivered	2025-02-04 20:00	9999-12-31 00:00:00

Q38. Which orders have a different latest status in the raw feed than in the orders table? DE MEDIUM

Approach: Latest row per key from the feed (P3), join to the target, compare NULL-safely.

```
WITH latest AS (
  SELECT order_id, status FROM (
    SELECT order_id, status, ROW_NUMBER() OVER (
      PARTITION BY order_id ORDER BY ingested_at DESC) AS rn
    FROM orders_raw) x
  WHERE rn = 1
)
SELECT o.order_id, l.status AS feed, o.status AS warehouse
FROM orders o
JOIN latest l ON l.order_id = o.order_id
WHERE NOT (l.status <=> o.status);
```

OUTPUT

order_id	feed	warehouse
102	placed	delivered

Order 102 was delivered in the warehouse but the feed never sent the update. Next question: is the feed late, or did the warehouse get the status from elsewhere?

PART 5 · PRACTICE AND REVISION

CONCEPT QUESTIONS, SHORT ANSWERS

Q&A

The no-keyboard part of the round. Answer in two sentences, then give one example from the store data.

WHERE vs HAVING?
WHERE filters rows before grouping; HAVING filters groups after. Aggregates only work in HAVING.

UNION vs UNION ALL?
UNION removes duplicates, which costs a sort or hash. UNION ALL keeps everything and is faster.

RANK vs DENSE_RANK vs ROW_NUMBER?
Ties: RANK repeats and skips, DENSE_RANK repeats without gaps, ROW_NUMBER never repeats (page 14).

DELETE vs TRUNCATE vs DROP?
DELETE removes chosen rows and can roll back. TRUNCATE empties the table fast. DROP removes the table itself.

COUNT(*) vs COUNT(col)?
COUNT(*) counts rows. COUNT(col) skips NULLs in that column.

Primary key vs unique key?
Both reject duplicates. A primary key is also NOT NULL, and a table has only one.

Clustered vs non-clustered index?
A clustered index is the table stored in key order (InnoDB's PK). A secondary index is a separate structure that points back to it.

Why can an index make a query slower?
For low-selectivity filters, jumping between index and table costs more than one sequential scan. Writes also pay to maintain it.

CTE vs temp table vs view?
CTE lives for one statement, temp table for the session, view forever as a saved query (page 12).

View vs materialized view?
A view runs its query every time. A materialized view stores the result and must be refreshed.

What is a correlated subquery?
A subquery that references the outer row, so it logically runs once per row (page 11).

Why does NOT IN return nothing?
A NULL in the list makes every comparison UNKNOWN. Use NOT EXISTS.

What is normalization, and why denormalize?
Normalization removes redundancy for safe writes. Warehouses denormalize to avoid joins in reads.

Star vs snowflake schema?
Star: fact plus flat dimensions. Snowflake: dimensions normalized into sub-tables. Star is simpler and faster to query.

Fact vs dimension table?
Facts hold measurable events at a grain (order lines). Dimensions describe them (customer, product, date).

What is the grain of a table?
What one row represents. Mixing grains in a join causes fan-out (page 10).

OLTP vs OLAP?
OLTP runs the application with small transactions. OLAP analyses history with large scans (page 31).

What does ACID mean?
Atomicity, consistency, isolation, durability (page 33).

What is an idempotent load?
One that gives the same result if run twice: upserts or partition overwrite, never blind appends.

SCD 1 vs SCD 2?
Type 1 overwrites and loses history. Type 2 adds a row per version with validity dates (page 35).

What is a surrogate key?
A generated id with no business meaning. Lets one customer have several SCD 2 rows.

What does QUALIFY do?
Filters on a window function result without a subquery. Snowflake, BigQuery, Databricks.

Partitioning vs clustering?
Partitioning splits data into separate chunks by a column so whole chunks are skipped. Clustering sorts data inside storage so blocks can be skipped.

How do you find duplicates?
GROUP BY the identifying columns HAVING COUNT(*) > 1. To remove them, ROW_NUMBER and keep 1 (page 22).

Can you use a SELECT alias in WHERE?
No. WHERE runs before SELECT creates the alias. Repeat the expression or wrap the query (page 5).

What is a window frame?
The rows a window function can see around the current row, set with ROWS or RANGE (page 15).

JOIN vs EXISTS for "has at least one"?
JOIN repeats the outer row once per match. EXISTS returns it once and can stop at the first match.

What is a covering index?
An index that contains every column a query needs, so the table itself is never read.

CHAR vs VARCHAR?
CHAR is fixed length and padded. VARCHAR stores only what you put in. Use CHAR for fixed codes.

What is a deadlock?
Two transactions each wait for a lock the other holds. The engine aborts one; the app retries it (page 33).

PART 5 · PRACTICE AND REVISION

THE ONE-PAGE REVISION SHEET

40

Read this the night before the interview. Everything on it is covered in detail somewhere in the previous 39 pages.

SYNTAX TO WRITE COLD — CHEAT SHEET

```
-- order: FROM > WHERE > GROUP BY > HAVING
--        > SELECT > DISTINCT > ORDER BY > LIMIT

-- top N per group                    (P1)
DENSE_RANK() OVER (PARTITION BY g
                   ORDER BY v DESC)

-- latest row per key                 (P3)
ROW_NUMBER() OVER (PARTITION BY id
                   ORDER BY ts DESC) = 1

-- running total, share of total      (P4)
SUM(x) OVER (ORDER BY t
             ROWS UNBOUNDED PRECEDING)
x / SUM(x) OVER ()

-- change vs previous                 (P6)
x - LAG(x) OVER (PARTITION BY id ORDER BY t)

-- islands                            (P7)
d - INTERVAL ROW_NUMBER() OVER (
      PARTITION BY id ORDER BY d) DAY

-- sessions                           (P8)
SUM(new_flag) OVER (PARTITION BY id
  ORDER BY t ROWS UNBOUNDED PRECEDING)

-- [a1,a2) overlaps [b1,b2)           (P16)
a1 < b2 AND b1 < a2
```

TRAP

The nine traps

- `NOT IN` with a NULL in the list returns nothing. Use NOT EXISTS.
- Right-table filters of a LEFT JOIN go in ON, not WHERE.
- Aggregate the many side before joining, or totals double.
- Default window frame is RANGE: ties share a running total.
- LAST_VALUE needs UNBOUNDED FOLLOWING.
- `= NULL` never matches. `<>` drops NULL rows too.
- Integer division: write 100.0 *, guard with NULLIF.
- ROW_NUMBER needs a unique tiebreaker to be repeatable.
- Functions on a filtered column kill index use.

APPROACH

The approach, short form

1 grain, 2 tables and keys, 3 sketch output, 4 name the pattern, 5 build in CTE layers, 6 break it with NULLs, ties, duplicates and date edges.

Dialect map: the same idea in four engines

Idea	MySQL 8	PostgreSQL	Snowflake	BigQuery
Month start	`DATE_FORMAT(d,'%Y-%m-01')`	`DATE_TRUNC('month', d)`	`DATE_TRUNC('month', d)`	`DATE_TRUNC(d, MONTH)`
Join strings in a group	`GROUP_CONCAT(x)`	`STRING_AGG(x, ',')`	`LISTAGG(x, ',')`	`STRING_AGG(x, ',')`
Filter on a window	CTE + WHERE	CTE + WHERE	`QUALIFY`	`QUALIFY`
Median	ROW_NUMBER trick (P15)	`PERCENTILE_CONT(0.5)`	`MEDIAN(x)`	`PERCENTILE_CONT(x, 0.5) OVER ()`
NULL-safe equal	`a <=> b`	`a IS NOT DISTINCT FROM b`	`EQUAL_NULL(a, b)`	`a IS NOT DISTINCT FROM b`
Upsert	`ON DUPLICATE KEY UPDATE`	`ON CONFLICT ... DO UPDATE`	`MERGE`	`MERGE`

A 4-week practice plan

Week	Pages	Do this	Done when
1	1 to 13	Load the store data into MySQL 8 locally. Retype every query in Part 1 and break each one on purpose (remove a WHERE, swap a JOIN).	You can explain each trap on pages 6, 10 and 11 without looking
2	14 to 19	Windows and recursion. Rewrite each practice answer two ways: with a window and without one.	You can predict RANK vs DENSE_RANK output before running
3	20 to 30	Two patterns a day. Solve the worked question from memory, then invent one new question per pattern on a different table.	You can name the pattern of any new question in 10 seconds
4	31 to 40	DE chapters for engineering roles. Then timed mocks: 3 questions in 45 minutes, talking through the 6 steps out loud.	You finish a HARD question in under 20 minutes

FROM DURGESH

One last thing. Interviewers rarely fail a candidate for a missing comma. They fail the answer that returns the wrong rows confidently. Check your grain, say your assumptions, and test against the edge cases. Everything else in this book is detail.

More notes, roadmaps and interview prep at **prepnplaced.com**.